

New Central Runbook

Configuration APIs

Version 2026.0331

DRAFT

The logo for HP Enterprise, featuring the letters "HP" in black and the stylized "E" in green.

Document Revision History

Date	Version	Author	Change
2/10/2026	2026.0210	Richard Wright	Initial Draft
2/27/2026	2026.0227	Richard Wright	Modified API Versions to /v1/
3/02/2026	2026.0302	Richard Wright	Multiple formatting and grammar edits
3/19/2026	2026.0319	Richard Wright	• Changed Title • Added Hierarchy Section • Fixed URIs for missing V1
3/20/2026	2026.0331	Richard Wright	• Fixed format on p105 • Added Delete WLAN section

Table of Contents

API NOTES	5
NEW CENTRAL HIERARCHY.....	6
AUTOMATION.....	9
DISCOVERY.....	10
Scope ID	10
Obtain Site List	12
Obtain Device List	13
Obtain Client List	15
Create Site	17
FOUNDATION.....	18
Create Admin User	18
System Administration Settings.....	20
NTP Server(s).....	23
DNS Server(s)	26
Aliases	28
NETWORK INFRASTRUCTURE	30
Hostname	30
VLANs	31
VLAN Interface (IPv4).....	35
Syslog Settings	41
SNMP Settings.....	46
Spanning Tree.....	50
SECURITY & AAA	57
AUTH Server (RADIUS / TACACS)	57
AUTH Server Group (RADIUS / TACACS)	60
AAA Profile	63
SWITCHING.....	70
Create VRF	70
Switch Port Profile	72
Configure Interface Settings	77
LAG (Port Channel)	87
Switch System Profile.....	91
WIRELESS	94
Underlay WLAN	96
Overlay WLAN	100
Update WLANs	105
Delete WLANs	106
VALIDATION	108
APPENDIX A – API ELEMENTS USED.....	112
APPENDIX B – NEW CENTRAL BASE URLS.....	114

This document aims to assist the understanding of configuration tasks associated with setting up and configuring the necessary elements for a wired and wireless network using APIs. The sections are divided into the following major areas:

0	Phase 0 – Discovery GET global Scope ID (required by all scope-map calls) Inventory sites, devices, and client connections Cache global_scope_id – it won't change for this tenant
1	Phase 1 – Foundation Admin User, System Administration (SSH/Console) NTP/DNS -> Apply at Global scope to all device types Aliases (ESSID, Hostname, etc.)
2	Phase 2 – Network Infrastructure Define Hostname VLANs (batch in a few POSTs); PVLAN if required VLAN Interface (IPv4/IPv6 addresses) Syslog – three separate profiles: Switch / AP / Gateway SNMP Spanning Tree (Core & Edge: MSTP RPVST)
3	Phase 3 – Security & AAA Auth Servers Auth Server Group AAA Profile
4	Phase 4 – Switching VRF if needed Switch Port Profile & Switch System Profile at Global Individual Switch Interface Settings (Device Scope) Port Channel (Device Scope) Switch System Profile
5	Phase 5 – Wireless Underlay WLAN (Bridge) Overlay WLAN (Tunnel) Scope to Global or Device Group for Zone-targeted SSIDs
6	Phase 6 – Validation Verify Config Distribution

API NOTES

As of the writing of this Runbook, the API monitoring and configuration locations are in transition and may be located at one of the following:

*** Note:** `/v1alpha1/` has been deprecated and replaced with `/v1/`

One Exception: `{{baseUrl}}/network-monitoring/v1alpha1/devices` is used to obtain Device List that includes the field `scopeId`, which is required to make configuration at the individual device-level

Monitoring: `{{baseUrl}}/network-monitoring/v1/{{element}}`

Configuration: `{{baseUrl}}/network-config/v1/{{element}}/{{profile-name}}`

Subsequent versions should be expected to update to `/v2/` ; `/v3/` ; etc.

The list of the Central Clusters (baseUrl) can be found in [Appendix B](#)

To list the configured elements of type `{{element}}` use:

GET `{{baseUrl}}/network-config/v1/{{element}}`

To list the settings for a specific element, use:

GET `{{baseUrl}}/network-config/v1/{{element}}/{{profile-name}}`

To create settings for a specific element, use:

POST `{{baseUrl}}/network-config/v1/{{element}}/{{profile-name}}`

To replace settings for a specific element, use:

PUT `{{baseUrl}}/network-config/v1/{{element}}/{{profile-name}}`

To delete settings for a specific element, use:

DELETE `{{baseUrl}}/network-config/v1/{{element}}/{{profile-name}}`

To map the configuration to a Device Type and Scope, use the API:

POST `{{baseUrl}}/network-config/v1/scope-maps`

To update a configuration for a specific element, use:

PATCH `{{baseUrl}}/network-config/v1/{{element}}/{{profile-name}}`

NEW CENTRAL HIERARCHY

In many cases it may be necessary to deploy configuration elements (such as WLANs) only to specific areas or devices. Central offers five levels of hierarchy that help direct where services and configurations will be applied. These levels are:

Global < Site Collections < Sites < Device Groups < Device Level

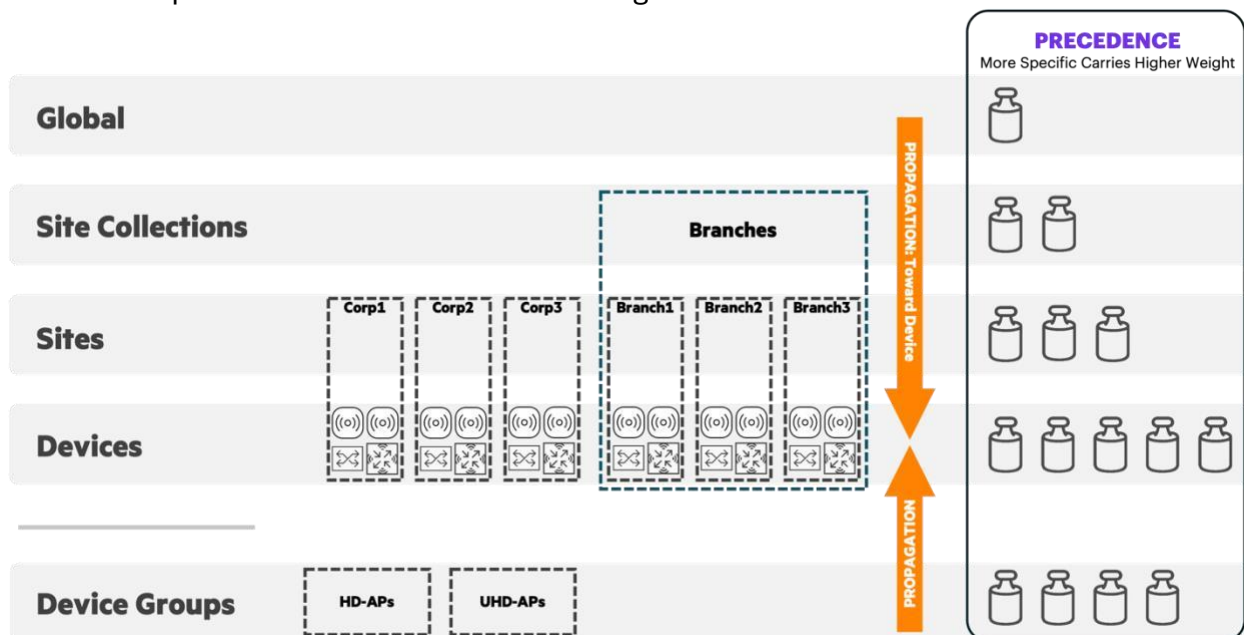
Global:	Config assigned to all devices/scopes
Site Collections:	Config assigned to devices in the Sites that are part of the collection
Sites:	Required in New Central. Config assigned to devices in the Site
Device Groups:	Config assigned to devices that are part of the group
Device:	Config assigned directly to the device

Currently, devices are placed into Classic Central Groups to assign New Central configuration. These Groups are the Device Groups in New Central.

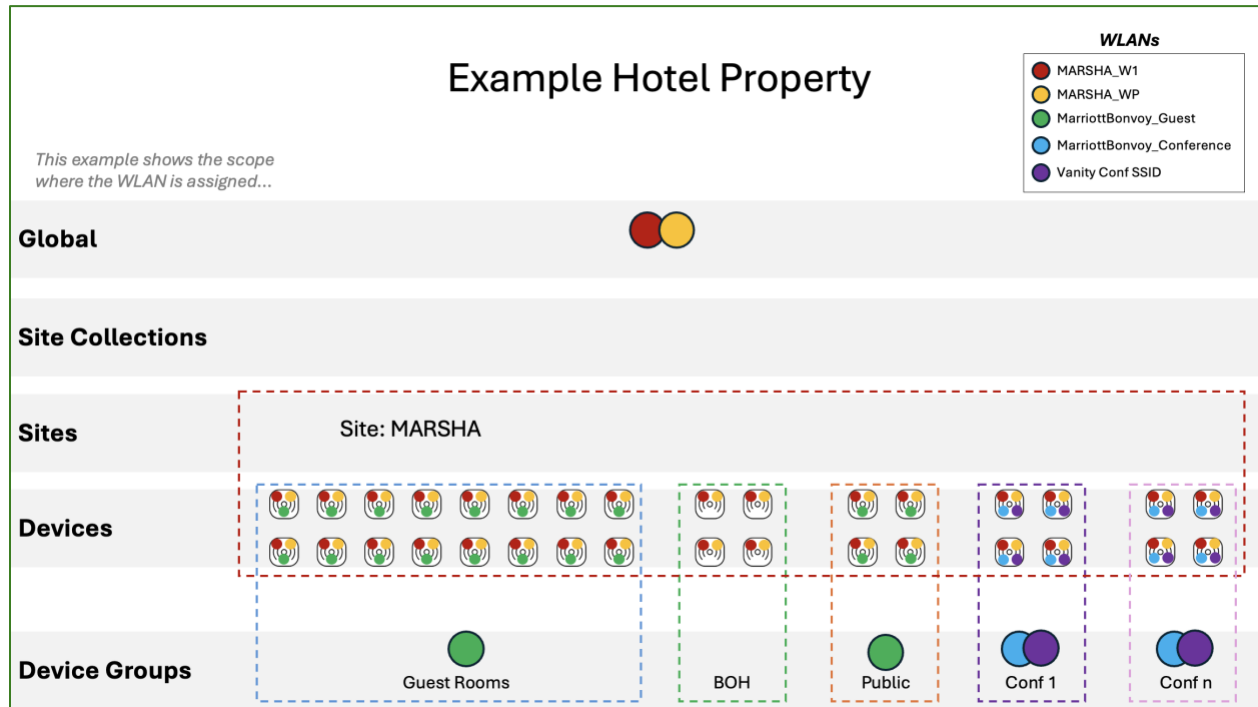
Services are assigned to the required scope level as listed above. Some services may apply globally to all devices (DNS) and should be assigned to the Global Scope, while others may be limited to specific scopes such as Conference WLANs (Device Group 1, Device Group 2, etc.)

In the case of some services (DNS, STP, Switch System, RF Profile, etc.), only one profile will be applied at the device. The configuration scope closest to the device will be applied.

Inheritance precedence follows the below diagram:



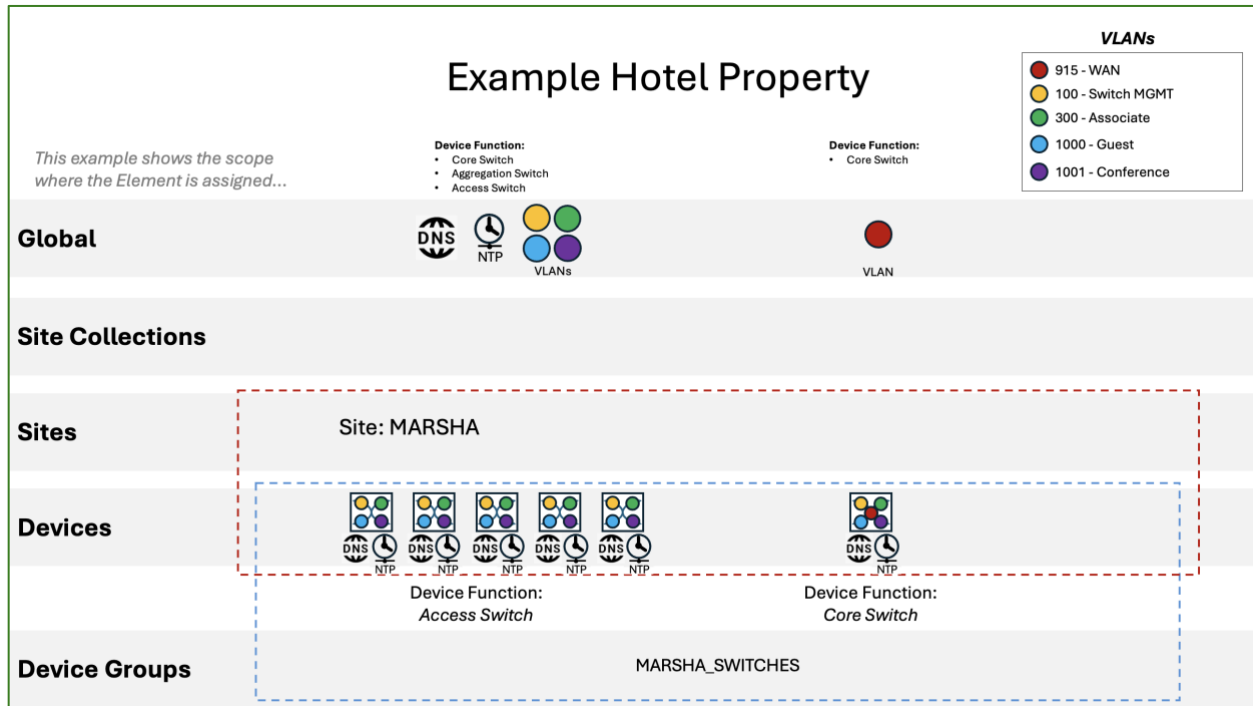
The below image shows an example of the scope where each WLAN could be assigned and how the inheritance flows. They are created in the Library then assigned to the appropriate scope-id and device function (CAMPUS_AP).



- WLAN: **MARSHA_W1**
Scope Assigned: Global
Where Available: All Access Points inherit
- WLAN: **MARSHA_WP**
Scope Assigned: Global
Where Available: All Access Points inherit
- WLAN: **MarriottBonvoy_Guest**
Scope Assigned: {Device Group Guest Rooms} | {Device Group Public}
Where Available: Only devices in the specified Device Groups inherit
- WLAN: **MarriottBonvoy_Conference**
Scope Assigned: (Device Group Conf1 | Device Group Conf n)
Where Available: Only devices in the specified Device Groups inherit
- WLAN: **Vanity_Conf**
Scope Assigned: (Device Group Conf1 | Device Group Conf n)
Where Available: Only devices in the specified Device Groups inherit

The assignment of device function (**CORE_SWITCH**, **AGGREGATION_SWITCH**, **ACCESS_SWITCH**) further limits an element's inheritance. When a configuration is associated with a specific device function, only devices that match the function will inherit the configuration. The assignment of the element within the hierarchy still applies to the inheritance rules.

The below image shows an example of scope assignment where different elements map to separate Device Functions, and how the inheritance flows. The elements are created in the Library then assigned to the appropriate scope-id and device function.



- | | |
|--|--|
| <ul style="list-style-type: none"> Element: DNS Scope Assigned: Global Where Available: All Switches inherit | <ul style="list-style-type: none"> Element: NTP Scope Assigned: Global Where Available: All Switches inherit |
| <ul style="list-style-type: none"> Element: VLAN 100 Scope Assigned: Global Where Available: All Switches inherit | <ul style="list-style-type: none"> Element: VLAN 300 Scope Assigned: Global Where Available: All Switches inherit |
| <ul style="list-style-type: none"> Element: VLAN 1000 Scope Assigned: Global Where Available: All Switches inherit | <ul style="list-style-type: none"> Element: VLAN 1001 Scope Assigned: Global Where Available: All Switches inherit |
| <ul style="list-style-type: none"> Element: VLAN 915 Scope Assigned: Global Where Available: Core Switches inherit | |

As shown in the above diagram

- Only switches defined as device function **CORE_SWITCH** will inherit **VLAN 915**
- All Switches will inherit the remaining VLANs as well as the DNS & NTP settings.

AUTOMATION

IMPORTANT INFORMATION:

The documentation provided within this document is solely for the convenience of partners to enable configuration of common network settings that can be replicated. For extensive API documentation, refer to the official HPE Aruba Networking Central API Documentation located here:



<https://developer.arubanetworks.com/new-central/docs/about>

<https://arubanetworking.hpe.com/techdocs/new-central/content/get-started/automation.htm>

The entire configuration can be automated and applied to the different levels of the hierarchy in Central. For this Runbook it is expected that **MOST** configuration will occur at the Library and applied to the Global context. In some cases, it will be necessary to apply configuration at a lower level in the hierarchy, such as Device Group for Conference APs, Device level for Hostname, etc. Follow best practices when applying the configuration.

For configurations that occur at the library, it is necessary to create the element and then map the configuration to the appropriate scope – generally Global. This will require the Element API to create the configuration then the Scope-Map API to assign to the appropriate scope and Device Function. For example:

- Create VLAN:
`POST {{baseUrl}}/network-config/v1/layer2-vlan`
- Scope Map VLAN:
`POST {{baseUrl}}/network-config/v1/scope-maps`
to `Global/CORE_SWITCH; AGG_SWITCH; ACCESS_SWITCH`

For certain configuration elements, creation occurs at a specific scope-id and Device Function. In these instances, only the creation API is used as it will be assigned during the creation of the element.

- Create Hostname:
`POST {{baseUrl}}/network-config/v1/aliases/sys_host_name?view-type=LOCAL&scope-id={{scope-id of device}}`

[Appendix A](#) lists the `{{elements}}` used in this document

[Appendix B](#) lists the HPE Aruba Central Cluster Base URLs

[\(return to TOC\)](#)

DISCOVERY

Scope ID

Retrieving the **global scope ID** is a two-step process. The first API call will return a list of sites or devices. From that list, select an object then execute the second API, scope-hierarchy. This API will return the details for the provided scope including the global scope ID.

In the below example, the API used obtains the list of sites:

STEP 1:

GET `{{baseUrl}}/network-config/v1/sites`

```
{
  "total": 8,
  "offset": null,
  "count": 8,
  "response": {
    "status": "SUCCESS",
    "message": "success",
    "code": "SCOPE-LIST-SITES-SUCCESS-001"
  },
  "items": [
    {
      "scopeName": "ATLCA",
      "scopeId": "23430278610",
      "longitude": -84.4561947,
      "city": "College Park",
      "id": "23430278610",
      "image": null,
      "country": "United States",
      "collectionId": null,
      "zipcode": "30337",
      "address": "2050 Sullivan Road",
      "latitude": 33.6239153,
      "state": "Georgia",
      "type": "network-config/sites",
      "collectionName": null,
      "deviceCount": 0,
      "timezone": {
        "rawOffset": -18000000,
        "timezoneId": "America/New_York",
        "timezoneName": "Eastern Standard Time"
      }
    }
  ],
  ## Output removed for space...
```

Once one of the sites (or devices) is selected, use the next API call to discover the hierarchy.

STEP 2:

GET `{{baseUrl}}/network-config/v1/hierarchy?scope-id={{scope-id}}&scope-type=site`

```
{
  "total": 1,
  "offset": null,
  "count": 1,
  "response": {
    "status": "SUCCESS",
    "message": "success",
    "code": "SCOPE-HIERARCHY-SUC-001"
  },
  "items": [
    {
      "id": "hierarchy0",
      "hierarchy": [
        {
          "childCount": 0,
          "scopeType": "site",
          "hostName": "",
          "scopeId": "23430278610",
          "scopeName": "ATLCA"
        },
        {
          "childCount": null,
          "scopeType": "org",
          "hostName": "",
          "scopeId": "370836161",
          "scopeName": ""
        }
      ],
      "type": "network-config/hierarchy"
    }
  ]
}
```

The global scope ID is identified in the "hierarchy" response with the 'scopeType' key with the value of "org". The global scope ID is the 'scopeId' value in that object. In the above example, **370836161** is the global scope-id.

Obtain Site List

Sites are mandatory in New Central. To obtain the Site Names and Site IDs, use this API. This API can be used to create sites using the **POST** method



For more information about the Site API visit:

<https://developer.arubanetworks.com/new-central-config/reference/createsitev1>

GET `{{baseUrl}}/network-monitoring/v1/sites`

Filters can be used to restrict the output to desired subset of devices

```
{
  "response": {
    "code": "SCOPE-LIST-SITES-SUCCESS-001",
    "message": "success",
    "status": "SUCCESS"
  },
  "offset": null,
  "count": 1,
  "total": 1,
  "items": [
    {
      "country": "United States",
      "scopeName": "ATLCA",
      "state": "Georgia",
      "timezone": {
        "timezoneName": "Eastern Standard Time",
        "rawOffset": -18000000,
        "timezoneId": "America/New_York"
      },
      "collectionId": null,
      "scopeId": "23421287680",
      "longitude": -84.4561947,
      "type": "network-config/sites",
      "collectionName": null,
      "deviceCount": 0,
      "latitude": 33.6239153,
      "zipcode": "30337",
      "id": "23421287680",
      "city": "College Park",
      "address": "2050 Sullivan Road",
      "image": null
    }
  ]
}
```

[\(return to TOC\)](#)

Obtain Device List

The Device List in New Central can be obtained as a full list or can be filtered.

GET `{{baseUrl}}/network-monitoring/v1/devices`

Filters can be used to restrict the output to desired subset of devices



For more information about the Device List API visit:

<https://developer.arubanetworks.com/new-central/reference/getdevicesv1>

For example:

GET `{{baseUrl}}/network-monitoring/v1/devices?filter=deviceName eq '{{deviceName}}'`

Where `{{deviceName}}` = DCARWSW02-01-MDF

```
{
  "next": null,
  "items": [
    {
      "ipv4": "10.4.3.130",
      "configStatus": "Synchronized with limitations",
      "ipv6": null,
      "configLastModifiedAt": "2026-02-13T15:16:33.967Z",
      "buildingId": null,
      "floorId": null,
      "notes": null,
      "clusterName": null,
      "role": null,
      "macAddress": "38:21:c7:ab:c5:90",
      "deviceType": "SWITCH",
      "deviceName": "DCARWSW02-01-MDF",
      "model": "CX-6100",
      "siteName": "P5-DCARW",
      "partNumber": "JL679A",
      "type": "network-monitoring/device-monitoring",
      "id": "TW23HDR19K",
      "persona": "Access Switch",
      "serialNumber": "TW23HDR19K",
      "deviceFunction": "Access Switch",
      "softwareVersion": "PL.10.16.1006",
      "deployment": "Standalone",
      "firmwareVersion": "PL.10.16.1006",
      "siteId": "27220376326",
      "status": "ONLINE",
      "uptimeInMillis": 727450,
      "lastSeenAt": null
    }
  ],
  "count": 1,
  "total": 1
}
```

The above API does not include the `scopeld` that is required to make configuration API calls. To obtain the device list that includes the `scopeld`, use the **network-config** API subset displayed on the following page.

*** Note:** at the time of this writing, the GET *network-config* devices API uses **v1alpha1**

GET `{{baseUrl}}/network-config/v1alpha1/devices?filter=name eq '{{deviceName}}'`

Where **filter** = `?filter=name eq 'DCARWSW02-01-MDF'` Response:

```
{
  "offset": null,
  "total": 1,
  "items": [
    {
      "model": "6100 12G CL4 2SFP+ 139W Swch (JL679A)",
      "scopeId": "25765472801",
      "id": "25765472801",
      "scopeName": "TW22KKW56V",
      "deployment": "Standalone",
      "deviceType": "SWITCH",
      "name": "DCARWSW02-01-MDF",
      "type": "network-config/devices",
      "persona": null,
      "deviceGroupName": "CNX",
      "siteId": "25964326768",
      "deviceGroupId": "834897312",
      "siteName": "P5-DCARW",
      "mac": "38:21:c7:ea:78:90",
      "ipv4": "10.4.3.130"
    }
  ],
  "count": 1,
  "response": {
    "status": "SUCCESS",
    "message": "success",
    "code": "SCOPE-DEVICE-SUC-001"
  }
}
```

[\(return to TOC\)](#)

Obtain Client List

The Client List in New Central can be obtained as a full list or can be filtered.

GET `{{baseUrl}}/network-monitoring/v1/clients`

Filters can be used to restrict the output to desired subset of devices



For more information about the Client List API visit:

<https://developer.arubanetworks.com/new-central/reference/listunifiedclients>

The below example used no filter but only one wired and one wireless client are shown.

Notice important client information **highlighted** in the response below:

```
{
  "next": null,
  "count": 2,
  "total": 2,
  "items": [
    {
      "clientName": "DCARWLT340686",
      "ipv4": "10.168.1.101",
      "ipv6": "fe80:0:0:0:3414:e073:b27f:889b",
      "type": "network-monitoring/client-monitoring",
      "connectedDeviceSerial": "SG77JW105Q",
      "connectedDeviceType": "SWITCH",
      "connectedTo": "DCARW-MDF-SW01",
      "lastSeenAt": "0",
      "id": "00:e0:4c:23:6b:32",
      "wlanName": null,
      "vlanId": "300",
      "vlanName": "300_ASSOCIATE_PCS",
      "clientFunction": "Operating System",
      "siteId": "25967473981",
      "clientVendor": "Debian/Ubuntu/Knoppix",
      "clientOperatingSystem": "Debian/Ubuntu/Knoppix Linux",
      "clientTags": "",
      "clientCategory": "Computing Systems",
      "clientManufacturer": "REALTEK SEMICONDUCTOR CORP.",
      "tunnelType": "None",
      "tunnelId": null,
      "connectedAt": "2026-02-13T15:53:21.145Z",
      "authenticationType": "",
      "status": "Connected",
      "keyManagement": null,
      "bssid": null,
      "radioMacAddress": null,
      "wirelessBand": null,
      "siteName": "DCARW",
      "wirelessSecurity": null,
      "wirelessChannel": null,
      "userName": "DCARWLT340686",
      "hostName": "DCARWLT340686",
    }
  ]
}
```

```

        "port": "1/1/7",
        "macAddress": "00:e0:4c:23:6b:32",
        "clientConnectionType": "Wired",
        "role": null,
        "clientCapabilities": null
    },
    {
        "clientName": "Richards-rMBP-5",
        "ipv4": "172.20.0.201",
        "ipv6": "fe80:0:0:0:1c0f:f8d1:9d1c:303d",
        "type": "network-monitoring/client-monitoring",
        "connectedDeviceSerial": "CNKSWNP0K9",
        "connectedDeviceType": "AP",
        "connectedTo": "DCARWAP01-01-OFFICE",
        "lastSeenAt": "0",
        "id": "80:e6:50:0f:91:50",
        "wlanName": "MAR_WP",
        "vlanId": "1017",
        "vlanName": null,
        "clientFunction": "Computer",
        "siteId": "25967473981",
        "clientVendor": "Apple",
        "clientOperatingSystem": "Apple Macbook",
        "clientTags": "",
        "clientCategory": "Computers & Servers",
        "clientManufacturer": "Apple, Inc.",
        "tunnelType": "None",
        "tunnelId": null,
        "connectedAt": "2026-02-13T15:55:47.720Z",
        "authenticationType": "",
        "status": "Connected",
        "keyManagement": "WPA2_PSK",
        "bssid": "cc:d0:83:cb:34:b1",
        "radioMacAddress": "cc:d0:83:cb:34:b0",
        "wirelessBand": "5 GHz",
        "siteName": "DCARW",
        "wirelessSecurity": "AES",
        "wirelessChannel": "44 (80 MHz)",
        "userName": "DCARWLT340686",
        "hostName": "Richards-rMBP-5",
        "port": null,
        "macAddress": "80:e6:50:0b:87:30",
        "clientConnectionType": "Wireless",
        "role": "MAR_WP",
        "clientCapabilities": "802.11ac"
    }
]
}

```

[\(return to TOC\)](#)

Create Site



For more information about the Site API visit:

<https://developer.arubanetworks.com/new-central-config/reference/createsitev1>

POST `{{baseUrl}}/network-monitoring/v1/sites`

Body

```
{
  "country": "United States",
  "name": "Test-Site2",
  "address": "2050 Sullivan Road",
  "city": "College Park",
  "state": "Georgia",
  "zipcode": "30337",
  "timezone": {
    "timezoneName": "Eastern Standard Time",
    "rawOffset": -18000000,
    "timezoneId": "America/New_York"
  }
}
```

Response

```
{
  "items": [
    "79987850123481088"
  ],
  "status": "SUCCESS",
  "code": "CREATED_SITE_SUCCESSFULLY",
  "message": "success"
}
```

In the above response, the value `"items": ["79987850123481088"]` is the Site scopeId

This may be used in subsequent API calls where it may be necessary to apply a configuration to the Site Hierarchy.

[\(return to TOC\)](#)

FOUNDATION

Create Admin User

Required: Create API & Scope-Map API

Admin Users can be locally created to allow access to devices if RADIUS/TACACS is not available. These can be multiple profiles with different usernames and access

POST `{{baseUrl}}/network-config/v1/management-users/{{name}}`

Body where:

`{{name}}` = sanchez

`{{password}}` = password1234

```
{
  "name": "sanchez",
  "username": "sanchez",
  "format": "CLEARTEXT",
  "password": "password1234",
  "rolename": "ROOT",
  "user-group": "administrators"
}
```

The configuration above is based on the following settings:

The screenshot shows a 'Create Profile' dialog box with the following fields:

- Name ***: Input field containing 'sanchez'.
- Username ***: Input field containing 'sanchez'.
- Password ***: Password input field with masked characters and a visibility toggle.
- Retype Password ***: Password input field with masked characters and a visibility toggle.
- Role ***: Dropdown menu with 'Admin' selected.

Map Admin User to Device Function(s) and Scope

Valid mappings are:

MOBILITY_GW	CAMPUS_AP
CORE_SWITCH (CX)	AOSS_CORE_SWITCH
AGG_SWITCH (CX)	AOSS_AGG_SWITCH
ACCESS_SWITCH (CX)	AOSS_ACCESS_SWITCH

In the following example, the Admin User is mapped to the GLOBAL Scope and the following Device Functions: **Access Switch, Aggregation Switch, Core Switch**

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where `{{name}}` = sanchez

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "management-users/sanchez"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "management-users/sanchez"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "management-users/sanchez"
    }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

Profiles	Roles & Policies	Named Objects	Services
Library > System > User Administration			
san ×			Create Profile
1 item			
Name	Assigned Device Function	Assigned Scope	
 sanchez	Core Switch Aggregation Switch Access Switch	Global	

[\(return to TOC\)](#)

System Administration Settings

Required: Create API & Scope-Map API

Use the System Administration settings to set the Console/SSH/Web access, login timeout, banner, and authentication priority.

POST `{{baseUrl}}/network-config/v1/local-management/{{sys_admin}}`

Body where `{{sys_admin}}` = `lsp_sys_admin`

```
{
  "name": "lsp_sys_admin",
  "description": "System Administration Profile with SSH enabled for
multiple VRFs",
  "webserver": {
    "enable": true,
    "enabled-vrfs": [
      "default"
    ]
  },
  "ssh-server-global-configs": {
    "enable": true,
    "enabled-vrfs": [
      "default",
      "mgmt"
    ]
  },
  "console": {
    "enable": true
  },
  "login": {
    "user-lockout": [
      {
        "session": "CONSOLE",
        "denylist-period": 60,
        "threshold": 3
      },
      {
        "session": "LOGIN",
        "denylist-period": 10,
        "threshold": 3
      }
    ]
  },
  "disable-kex": true,
  "login-session-timeout": 5,
  "banner-message": {
    "message-delimiter": "~",
    "text": "Do not attempt to connect if you are not authorized"
  }
}
```

The configuration above is based on the following settings:

Properties

References

Name *

lsp_sys_admin

Description

System Administration Profile with SSH enabled for multiple VRFs

General

☒ Console Access

Console Login Attempts

3

Console Lockout Time

60

Seconds

☒ SSH Access

SSH enable/disable is not applicable for Gateways

VRF

Selected VRF Profiles

VRF configuration is only applicable to CX

Encryption

Both

Encryption configuration is only applicable to Gateways

Authentication

☒ HMAC_SHA1

☒ HMAC_SHA1_96

☒ HMAC_SHA2_256

☒ Disable Weak DH Key Exchange...

Authentication configuration is only applicable to Gateways

☒ Web Access

VRF

default

VRF configuration is only applicable to CX

Login Session Timeout *

5

Minutes

Login Banner

Do not attempt to connect if you are not authorized

Login Retries

3

Retry Delay Seconds

10

Authentication

Authentication Type *

☒ Local

☐ RADIUS

☐ TACACS+

Administration Users

User Name	Role
sanchez	Admin
admin	Admin

☐ Switch Specific Parameters

Map System Administration (local-management) to Device Function(s) and Scope

Valid mappings are:

MOBILITY_GW	CAMPUS_AP
CORE_SWITCH (CX)	AOSS_CORE_SWITCH
AGG_SWITCH (CX)	AOSS_AGG_SWITCH
ACCESS_SWITCH (CX)	AOSS_ACCESS_SWITCH

In the following example, the Admin User is mapped to the GLOBAL Scope and the following Device Functions: **Access Switch, Aggregation Switch, Core Switch**

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where `{{sys_admin}}` = `lsp_sys_admin`

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "local-management/lsp_sys_admin"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "local-management/lsp_sys_admin"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "local-management/lsp_sys_admin"
    }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

Name	Assigned Device Function	Assigned Scope
 lsp_sys_admin System Administration Profile with SSH enabled for multiple...	Access Switch Aggregation Switch Core Switch	Global

[\(return to TOC\)](#)

NTP Server(s)

Required: Create API & Scope-Map API

NTP servers keep devices aligned with their system time, so logs are consistent within the site/property. When assigning NTP to a scope, only one NTP profile can be applied to a scope for each Device Function. One NTP profile can be applied at the Global scope, and another applied at a lower scope in the hierarchy – which will take precedence.

POST `{{baseUrl}}/network-config/v1/ntp/{{name}}`

Body where `{{name}}` = corporate_ntp

```
{
  "name": "corporate_ntp",
  "description": "Corp NTP Servers",
  "servers": [
    {
      "address": "10.10.10.111",
      "tx-mode": "IBURST",
      "iburst": "INITIAL",
      "max-poll": 6,
      "min-poll": 4,
      "prefer": true
    },
    {
      "address": "10.10.20.111",
      "tx-mode": "IBURST",
      "iburst": "INITIAL",
      "max-poll": 6,
      "min-poll": 4,
      "prefer": true
    }
  ]
}
```

The configuration above is based on the following settings:

NTP Server	
IP Address	Format
10.10.10.111	IPv4
10.10.20.111	IPv4

Below is an example of using the Public NTP Server:

POST `{{baseUrl}}/network-config/v1/ntp`

Body where `{{name}}` = my_ntp

```
{
  "name": "my_ntp",
  "description": "Public NTP Server",
  "servers": [
    {
      "address": "pool.ntp.org",
      "tx-mode": "IBURST",
      "iburst": "INITIAL",
    }
  ],
  "vrf": "default"
}
```

The configuration above is based on the following settings:

The screenshot shows the 'Properties' tab of a configuration interface. At the top, there are two tabs: 'Properties' (selected) and 'References'. Below the tabs, the 'Name' field is set to 'my_ntp'. The 'Description' field is set to 'Public NTP Server'. There is a checkbox for 'NTP Alias' which is currently unchecked. Below this, there is a section titled 'NTP Server' with a plus sign to add more servers. Under this section, there are two columns: 'IP Address' and 'Format'. The 'IP Address' field is set to 'pool.ntp.org' and the 'Format' field is set to 'FQDN'. Below these fields, there is a checkbox for 'Switch Specific Parameters' which is checked. At the bottom, there is a section titled 'Switch Parameters' with a 'VRF' field set to 'default'.

IP Address	Format
pool.ntp.org	FQDN

☒ Switch Specific Parameters

Switch Parameters

VRF *
default

Map NTP Server to Device Function(s) and Scope

Valid mappings are:

MOBILITY_GW	CAMPUS_AP
CORE_SWITCH (CX)	AOSS_CORE_SWITCH
AGG_SWITCH (CX)	AOSS_AGG_SWITCH
ACCESS_SWITCH (CX)	AOSS_ACCESS_SWITCH

In the following example, the NTP Server Profile is mapped to the GLOBAL Scope and the following Device Functions: **Access Switch, Aggregation Switch, Core Switch, Gateway**

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where `{{name}}` = my_ntp

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "ntp/my_ntp"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "ntp/my_ntp"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "ntp/my_ntp"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "MOBILITY_GW",
      "resource": "ntp/my_ntp"
    }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

Name	Assigned Device Function	Assigned Scope
 my_ntp Public NTP Server	Access Switch Aggregation Switch Core Switch...	Global

[\(return to TOC\)](#)

DNS Server(s)

Required: Create API & Scope-Map API

DNS servers allow devices to resolve FQDN entries, such as the Aruba Central management Cloud. When assigning DNS servers to a scope, only one DNS profile can be applied to a scope for each Device Function. Create multiple DNS Server entries in the profile if multiple are needed.

POST `{{baseUrl}}/network-config/v1/dns/{{name}}`

Body where `{{name}}` = public_dns

```
{
  "name": "public_dns",
  "resolver": [
    {
      "vrf": "default",
      "name-server": [
        {
          "ip": "1.1.1.1",
          "priority": 1
        },
        {
          "ip": "8.8.8.8",
          "priority": 2
        }
      ]
    }
  ]
}
```

The configuration above is based on the following settings:

Properties **References**

Name *

public_dns

Description

Domain Name

☐ DNS Alias

DNS Servers +

GW, AOS-S supports a maximum of 6, 4 DNS servers respectively and CX supports a maximum of 3 DNS servers per VRF.

IP Address	Format
1.1.1.1	IPv4
8.8.8.8	IPv4

Map DNS Server to Device Function(s) and Scope

Valid mappings are:

MOBILITY_GW	CAMPUS_AP
CORE_SWITCH (CX)	AOSS_CORE_SWITCH
AGG_SWITCH (CX)	AOSS_AGG_SWITCH
ACCESS_SWITCH (CX)	AOSS_ACCESS_SWITCH

In the following example, the DNS Profile is mapped to the GLOBAL Scope and the following Device Functions: **Access Switch, Aggregation Switch, Core Switch, Gateway**

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where `{{name}}` = my_ntp

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "dns/public_dns"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "dns/public_dns"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "dns/public_dns"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "MOBILITY_GW",
      "resource": "dns/public_dns"
    }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

Name	Assigned Device Function	Assigned Scope
 public_dns	Access Switch Aggregation Switch Mobility Gateway...	Global

[\(return to TOC\)](#)

Aliases

Required: Create API & Scope-Map API

Aliases are a way to make a common configuration that may have variations at a lower level. This enables creation of a single entity at a Global Scope (for instance) and reference an Alias that will be updated at lower Scopes. An example would be having a single WLAN configuration but creating an Alias for the ESSID-Name.

Types of Aliases that can be used in the configuration elements:

- | | | |
|----------------------------|-----------------------------|-----------------------|
| - Auth Server Address | - Auth Server Shared Secret | - DNS Server List |
| - Echo Source IP Address | - ESSID | - Host |
| - Identity Profile Control | - IPv4 Address | - IPv4 Prefix |
| - IPv4 System VLAN | - IPv6 Address | - IPv6 Prefix |
| - Network | - Network Destination | - NTP Server List |
| - Nexthop IPv4 Address | - Nexthop IPv6 Address | - Switchport |
| - System Host Name | - VLAN | - VLAN Active Gateway |
| - VLAN Description | - VLAN IPv4 Address | - VLAN IPv6 Address |
| - VLAN Voice Enable | | |

The example below will demonstrate creating an ESSID Alias.

POST `{{baseUri}}/network-config/v1/aliases/{{name}}`

Body where `{{name}}` = guest_alias

```
{
  "name": "guest_alias",
  "description": "Alias for Marriott Guest SSID",
  "type": "ALIAS_ESSID",
  "default-value": {
    "ssid-value": {
      "name": "MarriottBonvoy_Guest"
    }
  }
}
```

The configuration above is based on the following settings:

The screenshot shows a configuration form with two tabs: 'Properties' and 'References'. The 'Properties' tab is active. It contains a 'Name' field with the value 'guest_alias' and a 'Description' field with the value 'Alias for Marriott Guest SSID'. Below these is an 'Attributes' section with a 'Type' dropdown menu set to 'Essid' and an 'Essid Name' field with the value 'MarriottBonvoy_Guest'.

Map Alias to Device Function(s) and Scope

Valid mappings are (depending on Alias type):

MOBILITY_GW	CAMPUS_AP
CORE_SWITCH (CX)	AOSS_CORE_SWITCH
AGG_SWITCH (CX)	AOSS_AGG_SWITCH
ACCESS_SWITCH (CX)	AOSS_ACCESS_SWITCH

In the following example, Alias is mapped to the GLOBAL Scope and the following Device Functions: **Campus_AP**

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where `{{name}}` = my_ntp

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CAMPUS_AP",
      "resource": "aliases/guest_alias"
    }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

Name	Type	Assigned Device Function	Assigned Scope
 guest_alias Alias for Marriott Guest SSID	Essid	Campus Access Point	Global

NOTE: Application of Aliases at lower scopes is more difficult and requires careful planning. It may be best to create using the GUI, then retrieve the configuration using APIs. This will help devise the best approach to create and deploy aliases as intended.

[\(return to TOC\)](#)

NETWORK INFRASTRUCTURE

Hostname

Required: Create API at Device Scope

Creating the hostname is done at the specific Device scope and optionally using an alias.

POST `{{baseUrl}}/network-config/v1/aliases/sys_host_name?view-type=LOCAL&scope-id={{scope-id of device}}`

If the Alias already exists, it can be modified by **PATCH** or replaced with **PUT**

The settings `[view-type=LOCAL]` and `[scope-id]` will assign the scope of the setting to the device so later assignment is not required.

Body where `{{name}}` = sys_host_name

```
{
  "name": "sys_host_name",
  "type": "ALIAS_HOSTNAME",
  "default-value": {
    "hostname-value": {
      "hostname": "P5-CX6100-MDF"
    }
  }
}
```

The configuration above is based on the following settings:

Name *

sys_host_name

Description

Attributes

Type *

Hostname

Hostname *

P5-CX6100-MDF

The API settings above result in the following Device Function and scope assignment

Name	Type	Inherits From	Assigned Device Function	Assigned Scope	Overrides
✧ sys_host_name	Hostname	Self	Access Switch		○ -

[\(return to TOC\)](#)

VLANs

Required: Create API & Scope-Map API

VLANs can be created individually or define many within a single API call.

The below example demonstrates creating five VLANs mapping to the CX Access, Aggregation, and Core

Create Layer 2 VLAN

POST `{{baseUrl}}/network-config/v1/layer2-vlan`

Body:

```
{
  "l2-vlan": [
    {
      "name": "101_WAP_MGMT",
      "vlan": 101,
      "enable": true
    },
    {
      "name": "102_WAP_MGMT",
      "vlan": 102,
      "enable": true
    },
    {
      "name": "103_WAP_MGMT",
      "vlan": 103,
      "enable": true
    },
    {
      "name": "104_WAP_MGMT",
      "vlan": 104,
      "enable": true
    },
    {
      "name": "1000_GUEST_WIRELESS",
      "vlan": 1000,
      "enable": true
    }
  ]
}
```

This can be extended to include additional VLANs

To create **Private VLANs**, settings in the body of the VLAN element need to be modified. Private VLAN designation is only applicable to Switches, not Gateways or Access Points. When configuring a WLAN to terminate into a PVLAN, the WLAN will only need to reference the PVLAN ID. The switch will manage the isolation or community based on the VLAN ID.

Below displays the settings for a Primary, Isolated, and Community PVLAN:

VLAN 1000 - Primary, VLAN 2000 - Community, and VLAN 2100 - Isolated PVLAN

```
{
  "vlan": 1000,
  "name": "1000_PVLAN1000",
  "private-vlan-type": "PRIMARY",
  "voice-enable": false,
  "enable": true,
  "client-ip-tracker-enable": true,
  "igmp": {
    "snooping": false,
    "strict-version": false
  },
  "mld": {
    "snooping": false,
    "strict-version": false
  },
  "dhcpv4-snooping": {
    "enable": false
  },
  "dhcpv6-snooping": {
    "enable": false
  },
  "dynamic-arp-inspection": {
    "enable": false
  },
  "is-l3-vlan": null
},
{
  "vlan": 2000,
  "name": "2000_PVLAN2000",
  "private-vlan-type": "COMMUNITY",
  "private-vlan-association": 1000,
  "voice-enable": false,
  "enable": true,
  "client-ip-tracker-enable": true,
  "igmp": {
    "snooping": false,
    "strict-version": false
  },
  "mld": {
    "snooping": false,
    "strict-version": false
  },
  "dhcpv4-snooping": {
    "enable": false
  },
  "dhcpv6-snooping": {
    "enable": false
  }
}
```



```
    },
    "dynamic-arp-inspection": {
      "enable": false
    },
    "is-l3-vlan": null
  },
  {
    "vlan": 2100,
    "name": "2100_PVLAN2100",
    "private-vlan-type": "ISOLATED",
    "private-vlan-association": 1000,
    "voice-enable": false,
    "enable": true,
    "client-ip-tracker-enable": true,
    "igmp": {
      "snooping": false,
      "strict-version": false
    },
    "mld": {
      "snooping": false,
      "strict-version": false
    },
    "dhcpv4-snooping": {
      "enable": false
    },
    "dhcpv6-snooping": {
      "enable": false
    },
    "dynamic-arp-inspection": {
      "enable": false
    },
    "is-l3-vlan": null
  }
}
```

Map L2 VLAN to Device Function(s) and Scope

Valid mappings are:

MOBILITY_GW	
CORE_SWITCH (CX)	AOSS_CORE_SWITCH
AGG_SWITCH (CX)	AOSS_AGG_SWITCH
ACCESS_SWITCH (CX)	AOSS_ACCESS_SWITCH

In the following example, the VLAN is mapped to the GLOBAL Scope and the following Device Functions: **Access Switch, Aggregation Switch, Core Switch**
(not all VLANs scope-maps are displayed to preserve space)

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body:

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "layer2-vlan/{{vlanID}}"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "layer2-vlan/{{vlanID}}"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "layer2-vlan/{{vlanID}}"
    },
    { Remaining VLANs in configuration }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

VLAN ID	Name	Assigned Device Function	Assigned Scope
1000	1000_GUEST_WIRELESS	Core Switch Aggregation Switch Access Switch	Global
101	101_WAP_MGMT	Core Switch Aggregation Switch Access Switch	Global
102	102_WAP_MGMT	Core Switch Aggregation Switch Access Switch	Global
103	103_WAP_MGMT	Core Switch Aggregation Switch Access Switch	Global
104	104_WAP_MGMT	Aggregation Switch Access Switch Core Switch	Global

[\(return to TOC\)](#)

VLAN Interface (IPv4)

Required: Create API & Scope-Map API

To assign an IP address onto a VLAN, multiple components are required.

- Create L2-VLAN with basic needed settings
- **OPTIONAL** - Create IPv4 Address Alias
 - Create local to each device with unique settings
- Create Layer3 Interface
- Map all entries to Global Scope with correct Device Functions

The examples below show the process to create VLAN 200 with a unique IPv4 address on two separate switches: CX-6100 (Access Switch) & CX-6300 (Core Switch)

STEP 1: Create layer2-vlan **{{name}}** = 200

```
POST {{baseUrl}}/network-config/v1/layer2-vlan/{{name}}
{
  "vlan": 200,
  "name": "200_LAYER3",
  "description": "200_LAYER3",
  "enable": true,
  "client-ip-tracker-enable": true
}
```

STEP 2: Create aliases **{{name}}** = v_200_ipv4

This creates the Alias to be used in the dropdown, IPv4 Alias

```
POST {{baseUrl}}/network-config/v1/aliases/{{name}}
{
  "name": "v_200_ipv4",
  "description": "Alias for VLAN 200 - IPV4",
  "type": "ALIAS_IPV4_ADDRESS_TYPE"
}
```

STEP 2a: Create aliases **{{name}}** = v_200_ipv4 to LOCAL Devices with IP values

POST {{baseUrl}}/network-config/v1/aliases/{{name}}
?scope_id={{device1_scope}}&persona=ACCESS_SWITCH&object-type=LOCAL

```
{
  "name": "v_200_ipv4",
  "description": "Alias for VLAN 200 - IPV4",
  "type": "ALIAS_IPV4_ADDRESS_TYPE",
  "default-value": {
    "ipv4-address-type-value": {
      "ipv4-address": "10.99.99.10/24"
    }
  }
}
```

```

POST {{baseUrl}}/network-config/v1/aliases/{{name}}
?scope_id={{device2_scope}}&persona=CORE_SWITCH&object-type=LOCAL
{
  "name": "v_200_ipv4",
  "description": "Alias for VLAN 200 - IPV4",
  "type": "ALIAS_IPV4_ADDRESS_TYPE",
  "default-value": {
    "ipv4-address-type-value": {
      "ipv4-address": "10.99.99.20/24"
    }
  }
}

```

STEP 3: Create vlan-interfaces {{name}} = 200

This enables the checkbox “Enable L3”

```

POST {{baseUrl}}/network-config/v1/vlan-interfaces/{{name}}
{
  "id": 200,
  "is-valid": true,
  "enable": true,
  "ipv4": {
    "address-alias": "v_200_ipv4"
  }
}

```

STEP 4: Map the Global entries: layer2-vlan, aliases, vlan-interfaces

```

POST {{baseUrl}}/network-config/v1/scope-maps
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "layer2-vlan/200"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "layer2-vlan/200"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "layer2-vlan/200"
    }
  ]
}

```

POST `{{baseUrl}}/network-config/v1/scope-maps`

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "aliases/v_200_ipv4"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "aliases/v_200_ipv4"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "aliases/v_200_ipv4"
    }
  ]
}
```

POST `{{baseUrl}}/network-config/v1/scope-maps`

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "vlan-interfaces/200"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "vlan-interfaces/200"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "vlan-interfaces/200"
    }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

Library > VLANs & Networks > VLAN

VLAN ID	Name	Assigned Device Function	Assigned Scope
200 200_LAYER3	200_LAYER3	Core Switch Aggregation Switch Access Switch	Global

VLAN ID *

200

Name

200_LAYER3

ⓘ This configuration is only applicable to Switches

Description

200_LAYER3

☐ Description Alias

☒ Switch Specific Parameters

Switch Parameters

☒ Enable VLAN

Policy

None

☐ Access List

☐ DHCP v4 Snooping

☐ DHCP v6 Snooping

☐ Enable ARP Inspection

☐ Voice Enable Alias

☐ Voice

☒ Enable IP Client Tracker

IGMP

☐ Enable IGMP Snooping

Advanced

MLD

☐ Enable MLD Snooping

Advanced

☐ Gateway Specific Parameters

☒ Enable L3

☒ IPv4 Alias

v_200_ipv4

ⓘ Static

☐ IPv6 Alias

IPv6 Address Assignment *

☒ Static

☐ DHCP

ⓘ CX device doesn't support configuring DHCP-Client on more than one VLAN

☐ Relay to External

☒ Admin State

MTU

Library > Aliases

Name	Type	Assigned Device Function	Assigned Scope
 v_200_ipv4 Alias for VLAN 200 - IPV4	VLAN IPv4 Address	Access Switch Aggregation Switch Core Switch	Global

Name *

v_200_ipv4

Description

Alias for VLAN 200 - IPV4

Attributes

Type *

VLAN IPv4 Address

IP Address Assignment *

☒ Static

☐ DHCP

ⓘ CX device doesn't support configuring DHCP-Client on more than one VLAN

Device DCARW-MDF-SW01 > Aliases

DCARW-MDF-SW01 > Aliases
Core Switch

Q v_200 X

1 Item

Name	Type	Inherits From	Assigned Device Function
 v_200_ipv4 Alias for VLAN 200 - IPV4	VLAN IPv4 Address	Self	Core Switch

 This profile is overriding GLOBAL

Name *
v_200_ipv4

Description
Alias for VLAN 200 - IPV4

Attributes

Type *

VLAN IPv4 Address

IP Address Assignment *

☒ Static

☐ DHCP

ⓘ CX device doesn't support configuring DHCP-Client on more than one VLAN

IPv4 Address/Prefix *

10.99.99.10/24

Above shows override applied to the alias

Device P5-6100 > Aliases

P5-CX6100 > Aliases Access Switch

Q v_200 X

1 item

Name	Type	Inherits From	Assigned Device Function
v_200_ipv4 Alias for VLAN 200 - IPV4	VLAN IPv4 Address	Self	Access Switch

This profile is overriding GLOBAL

Name *
v_200_ipv4

Description
Alias for VLAN 200 - IPV4

Attributes

Type *
VLAN IPv4 Address

IP Address Assignment *
☒ Static
☐ DHCP

ⓘ CX device doesn't support configuring DHCP-Client on more than one VLAN

IPv4 Address/Prefix *
10.99.99.20/24

Above shows override applied to the alias

Below shows the VLAN interfaces with IP addresses on each switch

```
DCARW-MDF-SW01# show ip interface brief
Interface      IP Address      Interface Status
link/admin
1/1/10         10.4.2.1/24     down/up
vlan1          No Address      up/up
vlan100        10.4.3.131/25   up/up
vlan101        No Address      up/up
vlan200       10.99.99.10/24  down/up
DCARW-MDF-SW01#
```

```
P5-CX6100# show ip interface brief
Interface      IP Address      Interface Status
link/admin
vlan1          No Address      up/up
vlan100        10.4.3.130/25   up/up
vlan101        No Address      up/up
vlan200       10.99.99.20/24  up/up
P5-CX6100#
```

[\(return to TOC\)](#)

Syslog Settings

Required: Create API & Scope-Map API

Syslog enables sending Alerts and Traps to a centralized location for alert and event correlation. The configuration for Access Points, Switches, and Gateways utilize different formats. Below are examples for each:

SYSLOG: SWITCH

POST `{{baseUrl}}/network-config/v1/logging/{{name}}`

Body where `{{name}}` = syslog_server

```
{
  "name": "syslog_server",
  "remote-syslog": [
    {
      "host": "192.168.224.2",
      "logging-syslog-enable": true,
      "severity": "WARNING",
      "vrf": "default"
    }
  ],
  "destination": "SYSLOG",
  "severity": "WARNING"
}
```

The configuration above is based on the following settings:

Name *

Description

Device Type *
☐ AP
☒ Switch
☐ Gateway

Debug Syslog Logging Level

Syslog Server +

FQDN/IP Address	Logging Level	VRF	Port
192.168.224.2	WARNING	default	-

Server Address Format *
☐ FQDN
☒ IP Address

IP Address *

Logging Level *

VRF *

Transport Protocol *

Port

SYSLOG: ACCESS POINT

POST `{{baseUrl}}/network-config/v1/logging/{{name}}`

Body where `{{name}}` = `syslog_server_ap`

```
{
  "name": "syslog_server_ap",
  "remote-syslog": [
    {
      "host": "192.168.224.2"
    }
  ],
  "severity": "WARNING"
}
```

The configuration above is based on the following settings:

The image displays two screenshots from the HPE Central Configuration API Runbook. The left screenshot shows the 'Syslog Server' configuration page. It includes a 'Name' field with the value 'syslog_server_ap', a 'Description' field, a 'Device Type' section with radio buttons for 'AP' (selected), 'Switch', and 'Gateway', a 'Debug Syslog Logging Level' dropdown set to 'Warning', and a summary section showing the 'IP Address' as '192.168.224.2'. The right screenshot shows the 'Edit Logging Server' dialog box, which has a title bar with a back arrow, the title 'Edit Logging Server', and a close button (X). The main content area shows the 'IP Address' field with the value '192.168.224.2'.

SYSLOG: GATEWAY

POST `{{baseUrl}}/network-config/v1/logging/{{name}}`

Body where `{{name}}` = `syslog_server_gw`

```
{
  "name": "syslog_server_gw",
  "remote-syslog": [
    {
      "host": "192.168.224.2~SYSTEM",
      "severity": "WARNING",
      "format": "CEF",
      "tls": false,
      "local-facility": "LOCAL0"
    }
  ]
}
```

The configuration above is based on the following settings:

Name *

syslog_server_gw

Description

Device Type *

☐ AP
 ☐ Switch
 ☒ Gateway

Syslog Server

+

IP Address	Category	Facility	Level	TLS
192.168.2...	SYST...	LOC...	WARN...	Disa...

IP Address *

192.168.224.2

Category

System

Logging Facility

Local0

Logging Level

Warning

☐ TLS

Format

CEF

Port

Map Syslog Server to Device Function(s) and Scope

Each Device Type uses a different Syslog configuration

MOBILITY_GW	CAMPUS_AP
CORE_SWITCH (CX)	AOSS_CORE_SWITCH
AGG_SWITCH (CX)	AOSS_AGG_SWITCH
ACCESS_SWITCH (CX)	AOSS_ACCESS_SWITCH

In the following example, the correct Syslog Server is mapped to the GLOBAL Scope and the appropriate Device Functions:

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where name is matched for each Device Function:

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "logging/syslog_server"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "logging/syslog_server"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "logging/syslog_server"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "MOBILITY_GW",
      "resource": "logging/syslog_server_ap"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CAMPUS_AP",
      "resource": "logging/syslog_server_gw"
    }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

Name	Assigned Device Function	Assigned Scope
 syslog_server	Aggregation Switch Core Switch Access Switch	Global
 syslog_server_ap	Campus Access Point	Global
 syslog_server_gw	Mobility Gateway	Global

[\(return to TOC\)](#)

SNMP Settings

Required: Create API & Scope-Map API

SNMP servers keep devices aligned with their system time, so logs are consistent within the site/property. Only one SNMP profile can be applied to a scope for each Device Function. If one SNMP profile is applied at the Global scope, and a different one applied at a lower scope in the hierarchy – the lower will take precedence.

POST `{{baseUrl}}/network-config/v1/snmp/{{name}}`

Body where `{{name}}` = XYZ-SNMP

```
{
  "name": "XYZ-SNMP",
  "description": "SNMP for devices at property",
  "v2c-enable": true,
  "v3-enable": false,
  "community": [
    {
      "community": "678!ZJsaw*@0091"
    }
  ],
  "vrf": [
    "default"
  ],
  "notification-receiver": [
    {
      "host-port-version-receiver-name-vrf":
        "192.168.224.1~162~V2C~678!ZJsaw*@0091~default",
      "host": "192.168.224.1",
      "port": 162,
      "version": "V2C",
      "receiver-name": "678!ZJsaw*@0091",
      "inform": true,
      "vrf": "default"
    },
    {
      "host-port-version-receiver-name-vrf":
        "192.168.10.252~162~V2C~678!ZJsaw*@0091~default",
      "host": "192.168.10.252",
      "port": 162,
      "version": "V2C",
      "receiver-name": "678!ZJsaw*@0091",
      "inform": true,
      "vrf": "default"
    }
  ]
}
```

The configuration above is based on the following settings:

PropertiesReferences

Name *

XYZ-SNMP

Description

SNMP for devices at property

Version *

☒ V2C

☐ V3

Community Strings

Name

+

678!ZJsaw*@0091

Trap Receivers

Version

IP Address

Community/User

V2C

192.168.224.1

678!ZJsaw*@0091

V2C

192.168.10.252

678!ZJsaw*@0091

☒ AOS-CX specific parameter

VRF

default

Map SNMP Profile to Device Function(s) and Scope

Valid mappings are:

MOBILITY_GW	
CORE_SWITCH (CX)	AOSS_CORE_SWITCH
AGG_SWITCH (CX)	AOSS_AGG_SWITCH
ACCESS_SWITCH (CX)	AOSS_ACCESS_SWITCH

In the following example, the SNMP Profile is mapped to the GLOBAL Scope and the following Device Functions: **Access Switch, Aggregation Switch, Core Switch, Gateway, Access Point**

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body:

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "snmp/XYZ-SNMP"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "snmp/XYZ-SNMP"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "snmp/XYZ-SNMP"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "MOBILITY_GW",
      "resource": "snmp/XYZ-SNMP"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CAMPUS_AP",
      "resource": "snmp/XYZ-SNMP"
    }
  ]
}
```


The API settings above result in the following Device Function and scope assignment

Name	Assigned Device Function	Assigned Scope
 XYZ-SNMP SNMP for devices at property	Core Switch Aggregation Switch Campus Access Point...	Global

[\(return to TOC\)](#)

Spanning Tree

Required: Create API & Scope-Map API

When creating Spanning Tree for the network, the configuration allows creation for either Switches or Gateways. When creating the profile, select the device type to which the profile will be applied. Choose either MSTP or RPVST mode according to the requirements. A device can only have one STP profile applied. Refer to the hierarchy inheritance to determine where to apply the profile.

The examples below will show simple options of Core & Edge, MSTP & RSTP configurations with different priorities. Multiple options (forward-delay, priority, cost, etc.) can be configured as needed. The examples depict a few of these options but is not a comprehensive guide.

STP: Core Switch MSTP

POST `{{baseUrl}}/network-config/v1/stp/{{name}}`

Body where `{{name}}` = core_mstp & priority = 4

```
{
  "name": "core_mstp",
  "enable": true,
  "mode": "MSTP",
  "forward-delay": 15,
  "hello-time": 2,
  "max-age": 20,
  "priority": 4,
  "rpvst-auto-enable": false
}
```

The Core **MSTP** configuration above is based on the following settings:

STP
Manage STP Profile.

Name *
core_mstp

Description

☒ Enable STP

Device
☒ Switch
☐ Gateway

Mode
☒ MSTP
☐ RPVST

Advanced ▾

Default Instance Priority
0 (Highest) 15 (Lowest)

4 (16384)

STP: Core Switch RPVST

POST `{{baseUrl}}/network-config/v1/stp/{{name}}`

Body where `{{name}}` = core_rpvst & priority = 4

```
{
  "name": "core_rpvst",
  "enable": true,
  "mode": "RPVST",
  "extend-system-id": true,
  "ignore-pvid-inconsistency": false,
  "pathcost-type": "COST_TYPE_LONG",
  "rpvst-mstp-interconnect-vlan": 1,
  "transmit-hold-count": 6,
  "rpvst-auto-enable": true,
  "rpvst-auto-priority": 4
}
```

The Core **RPVST** configuration above is based on the following settings:

STP

Manage STP Profile.

Name *

core_rpvst

Description

☒ Enable STP

Device

☒ Switch

☐ Gateway

Mode

☐ MSTP

☒ RPVST

Advanced ▼

RPVST Mode

☒ Auto RPVST (Include VLANs)

☐ Manual RPVST

Auto RPVST Priority

0 (Highest) 15 (Lowest)

4 (16384)

Edge Switch MSTP

POST `{{baseUrl}}/network-config/v1/stp/{{name}}`

Body where `{{name}}` = edge_mstp & priority = 8

```
{
  "name": "edge_mstp",
  "enable": true,
  "config-name": "REGION-1",
  "config-revision": 0,
  "mode": "MSTP",
  "forward-delay": 15,
  "hello-time": 2,
  "max-age": 20,
  "max-hops": 20,
  "pathcost-type": "COST_TYPE_LONG",
  "priority": 8,
  "transmit-hold-count": 6,
  "traps": [
    "ERRANT_BPDU",
    "LOOP_GUARD",
    "NEW_ROOT",
    "ROOT_GUARD"
  ],
  "topology-change-trap": false
}
```

The Edge **MSTP** configuration above is based on the following settings:

STP

Manage STP Profile.

Name *

edge_mstp

Description

☒ Enable STP

Device

☒ Switch

☐ Gateway

Mode

☒ MSTP

☐ RPVST

Advanced

BPDU Guard Enable Timer(in seconds)

Path Cost Type

☒ LONG

☐ SHORT

Transmit Hold Count

1 (Highest) 10 (Lowest)

6

Traps

☒ Errant BPDU

☒ Loop Guard Inconsistency

☒ New Root

☒ Root Guard Inconsistency

Region

The default value is a switch MAC address.

REGION-1

Config Revision

0

Forward Delay

4 seconds 30 seconds

15 seconds

Hello Timer

2 seconds 10 seconds

2 seconds

Maximum Age

6 seconds 40 seconds

20 seconds

Maximum Hops

1 (Highest) 40 (Lowest)

20

Reset above timers to default

☐ Reset timers to default

☐ Topology Change Trap for Default...

Default Instance Priority

0 (Highest) 15 (Lowest)

8 (32768)

Edge Switch RPVST

POST `{{baseUrl}}/network-config/v1/stp/{{name}}`

Body where `{{name}}` = edge_rpvst & priority = 8

```
{
  "name": "edge_rpvst",
  "enable": true,
  "mode": "RPVST",
  "extend-system-id": true,
  "ignore-pvid-inconsistency": false,
  "pathcost-type": "COST_TYPE_LONG",
  "rpvst-mstp-interconnect-vlan": 1,
  "transmit-hold-count": 6,
  "traps": [
    "ERRANT_BPDU",
    "LOOP_GUARD",
    "NEW_ROOT",
    "ROOT_GUARD"
  ],
  "rpvst-auto-enable": true,
  "rpvst-auto-priority": 8
}
```

The Edge **RPVST** configuration above is based on the following settings:

STP

Manage STP Profile.

Name *

edge_rpvst

Description

☒ Enable STP

Device

☒ Switch

☐ Gateway

Mode

☐ MSTP

☒ RPVST

Advanced

BPDU Guard Enable Timer(in seconds)

Path Cost Type

☒ LONG

☐ SHORT

Transmit Hold Count

1 (Highest) 10 (Lowest)

6

Traps

☒ Errant BPDU

☒ Loop Guard Inconsistency

☒ New Root

☒ Root Guard Inconsistency

☐ Ignore PVID Inconsistency

☒ Extend System ID

RPVST MSTP Interconnect-VLAN

1

RPVST Mode

☒ Auto RPVST (Include VLANs)

☐ Manual RPVST

Auto RPVST Priority

0 (Highest) 15 (Lowest)

8 (32768)

Map STP Profiles to Device Function(s) and Scope

Based on the previous configurations, valid mappings are:

CORE_SWITCH (CX)	AOSS_CORE_SWITCH
AGG_SWITCH (CX)	AOSS_AGG_SWITCH
ACCESS_SWITCH (CX)	AOSS_ACCESS_SWITCH

In the following example, the **MSTP** STP Profiles are mapped to the GLOBAL Scope and the following Device Functions: **Access Switch, Aggregation Switch, Core Switch**

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where:

CORE/AGG Switch `{{name}}` = core_mstp

ACCESS Switch `{{name}}` = edge_mstp

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "stp/core_mstp"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "stp/core_mstp"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "stp/edge_mstp"
    }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

Name	Assigned Device Function	Assigned Scope
 core_mstp	Aggregation Switch Core Switch	Global
 core_rpvst	-	-
 edge_mstp	Access Switch	Global
 edge_rpvst	-	-

[\(return to TOC\)](#)

SECURITY & AAA

AUTH Server (RADIUS / TACACS)

Required: Create API & Scope-Map API

RADIUS Servers are required to authenticate onto the network when the interface or WLAN is untrusted. TACACS can also be created using the field - "type": "TACACS"

To create a RADIUS Server, two API calls are needed:

- Create AUTH Server
- Map the AUTH Server to Device Function and Scope

POST `{{baseUrl}}/network-config/v1/auth-servers/{{server_name}}`

Body where `{{server_name}}` = RADICAL_Server

```
{
  "name": "RADICAL_Server",
  "type": "RADIUS",
  "auth-server-address": "192.168.200.200",
  "enable-radsec": false,
  "auth-port": 1812,
  "acct-port": 1813,
  "clearpass": false,
  "shared-secret-config": {
    "secret-type": "PLAIN_TEXT",
    "plaintext-value": "supersecretPassCode"
  }
}
```

The configuration above is based on the following settings:

The screenshot shows the configuration form for a RADIUS server named 'RADICAL_Server'. The form is divided into two tabs: 'Properties' and 'References'. The 'Properties' tab is active, showing various configuration fields. The 'Name' field is set to 'RADICAL_Server'. The 'Description' field is empty. The 'Server Type' is set to 'RADIUS'. The 'Auth Server Mode' is set to 'RADIUS' (selected with a radio button). The 'Server Address Alias' is set to '192.168.200.200'. The 'Shared Secret Alias' is set to 'supersecretPassCode'. The 'Authentication Port' is set to '1812' and the 'Accounting Port' is set to '1813'. The 'Advanced' checkbox is unchecked.

Map AUTH Server to Device Function(s) and Scope

Valid mappings are:

MOBILITY_GW	CAMPUS_AP
CORE_SWITCH (CX)	AOSS_CORE_SWITCH
AGG_SWITCH (CX)	AOSS_AGG_SWITCH
ACCESS_SWITCH (CX)	AOSS_ACCESS_SWITCH

In the following example, the Auth Server is mapped to the GLOBAL Scope and the following Device Functions: **Access Switch, Aggregation Switch, Core Switch, Mobility Gateway, Campus Access Point**

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where `{{server_name}}` = RADICAL_Server

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "auth-servers/RADICAL_Server",
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "auth-servers/RADICAL_Server",
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "auth-servers/RADICAL_Server",
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "MOBILITY_GW",
      "resource": "auth-servers/RADICAL_Server",
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CAMPUS_AP",
      "resource": "auth-servers/RADICAL_Server",
    }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

Q Search

7 items

Campus Access Point

Aggregation Switch

Core Switch

Access Switch

Mobility Gateway

Create Profile

Name	Type	Assigned Scope
RADICAL_Server	RADIUS	Campus Access Point Aggregation Switch Core Switch... Global

...

[\(return to TOC\)](#)

AUTH Server Group (RADIUS / TACACS)

Required: Create API & Scope-Map API

RADIUS Server Groups are required to authenticate onto the network. Authentication is directed to a Server Group that may contain one or more servers. TACACS Groups can also be created using the field - "type": "TACACS"

To create a RADIUS Server Group, two API calls are needed:

- Create AUTH Server Group
- Map the AUTH Server Group to Device Function and Scope

POST `{{baseUrl}}/network-config/v1/server-groups/{{server_group_name}}`

Body where:

`{{server_group_name}}` = RADICAL_Server_Group

`{{server_name}}` = RADICAL_Server

```
{
  "name": "RADICAL_Server_Group",
  "type": "RADIUS",
  "load-balance": false,
  "fail-through": false,
  "servers": [
    {
      "server-name": "RADICAL_Server",
      "position": 1
    }
  ]
}
```

The configuration above is based on the following settings:

The screenshot shows a configuration form with two tabs: 'Properties' and 'References'. The 'Properties' tab is selected. It contains the following fields and options:

- Name ***: A text input field containing 'RADICAL_Server_Group'.
- Description**: A large text area for additional information.
- Server Type**: A dropdown menu set to 'RADIUS'.
- Authentication Servers ***: A dropdown menu set to 'RADICAL_Server'.
- Name**: A text input field containing 'RADICAL_Server'.
- Gateway Specific Parameters**: A checkbox that is unchecked.
- Switch Specific Parameters**: A checkbox that is unchecked.
- Advanced**: A checkbox that is unchecked.

Map AUTH Server Group to Device Function(s) and Scope

Valid mappings are:

MOBILITY_GW	CAMPUS_AP
CORE_SWITCH (CX)	AOSS_CORE_SWITCH
AGG_SWITCH (CX)	AOSS_AGG_SWITCH
ACCESS_SWITCH (CX)	AOSS_ACCESS_SWITCH

In the following example, the Auth Server Group is mapped to the GLOBAL Scope and the following Device Functions: **Access Switch, Aggregation Switch, Core Switch, Mobility Gateway, Campus Access Point**

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where `{{server_group_name}}` = RADICAL_Server_Group

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "server-groups/RADICAL_Server_Group"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "server-groups/RADICAL_Server_Group"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "server-groups/RADICAL_Server_Group"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "MOBILITY_GW",
      "resource": "server-groups/RADICAL_Server_Group"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CAMPUS_AP",
      "resource": "server-groups/RADICAL_Server_Group"
    }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

Name	Type	Servers	Assigned Device Function	Assigned Scope
>  RADICAL_Server_Group	RADIUS	1	Aggregation Switch, Core Switch, Mobility Gateway, Access Switch, Campus Access Point	Global

[\(return to TOC\)](#)

AAA Profile

Required: Create API & Scope-Map API

AAA Profiles provide the settings used to authenticate clients onto the network. This profile relies on [dot1xauth profile](#), [macauth profile](#) and the RADIUS Server Group which must first be created prior creating the AAA Profile. The GUI configuration creates all necessary profiles in the AAA dialogue, but they are separated when configuring via API.

Dot1X Auth Profile

Required: Create API & Scope-Map API

POST [{{baseUrl}}](#)/network-config/v1/dot1xauth/{{name}}

Body where **{{name}}** = my_wired_aaa_flood

```
{
  "name": "my_wired_aaa_flood",
  "enable": true,
  "eapol-max-requests": 1,
  "eapol-timeout": 5,
  "reauth-enable": true,
  "reauth-period": 28800,
  "cached-reauth-enable": false
}
```

***NOTE:** In the GUI this is created in the AAA Authentication

MAC Auth Profile

Required: Create API & Scope-Map API

POST [{{baseUrl}}](#)/network-config/v1/macauth/{{name}}

Body where **{{name}}** = my_wired_aaa_flood

```
{
  "name": "my_wired_aaa_flood",
  "enable": true,
  "max-retries": 0,
  "reauth-enable": true,
  "reauth-period": 28800,
  "cached-reauth-enable": false
}
```

***NOTE:** In the GUI this is created in the AAA Authentication

AAA Profile

Required: Create dot1x, macauth, & AAA API & Scope-Map each API

This profile combines the **dot1xauth**, **macauth**, and **server_groups** profiles into the AAA profile to be assigned to switch interfaces to provide authentication.

POST `{{baseUrl}}/network-config/v1/aaa-profile/{{aaa_profile_name}}`

Body where:

```

{{aaa_profile_name}} = my_wired_aaa_flood      {{dot1x_auth_profile}} = my_wired_aaa_flood
{{mac_auth_profile}} = my_wired_aaa_flood      {{dot1x_server_group}} = RADIUS-1-Group
{{macauth_server_group}} = RADIUS-1-Group      {{acct_group}} = RADIUS-1-Group
{
  "name": "my_wired_aaa_flood",
  "description": "Port Authentication with Flood",
  "authentication": {
    "dot1x-auth": "my_wired_aaa_flood",
    "mac-auth": "my_wired_aaa_flood",
    "dot1xauth-server-group": "RADIUS-1-Group",
    "macauth-server-group": "RADIUS-1-Group",
    "onboarding-precedence": [
      "DEVICE_PROFILE",
      "AAA"
    ],
    "concurrent-onboarding": true,
    "cloud-auth": false,
    "auth-mode": "CLIENT_MODE",
    "allow-flood-traffic": true,
    "client-limit": 2,
    "bpdu-auth": {
      "lldp": true,
      "cdp": true,
      "mac-type": "CHASSIS_MAC"
    },
    "bpdu-bypass": {
      "lldp": true,
      "cdp": false
    },
    "bpdu-proxy-logout": {
      "cdp": false
    },
    "device-profile-secure": false
  },
  "authorization": {
    "radius-override-enable": false
  },
  "security-violation": {
    "shutdown-recovery-enable": false
  },
  "acct-server-group": "RADIUS-1-Group",
  "interim-accounting": false
}

```


The configuration above is based on the following settings:

Name * my_wired_aaa_flood Description Port Authentication with Flood Authentication Protocol * Concurrent Client Limit 2 802.1X Authentication Server Group RADIUS-1-Group MAC Authentication Server Group RADIUS-1-Group Authentication Parameters ☒ Switch Specific Parameters ☒ Allow Flood Traffic Authentication Mode Client Mode Client Auto Logoff Select ☐ Secure Device Profile Onboarding Precedence Device Profile, then AAA	LLDP Support ☒ Allow LLDP Authentication ☒ Chassis MAC ☐ Interface MAC ☒ Allow LLDP Bypass CDP Support ☒ Allow CDP Authentication ☐ Allow CDP Bypass ☐ Allow CDP Proxy Logoff Security Violation Action Select Authorization Parameters ☐ Switch Specific Parameters Accounting ☐ Interim RADIUS Accounting Accounting Server Group RADIUS-1-Group
--	---

Below are the sections of the AAA GUI that create the dot1xauth and macauth profiles

802.1X Parameters

Max Authentication Failure

☒ Reauthentication
Reauthentication Interval
 seconds
Quiet Period
 seconds
☒ Switch Specific Parameters

Switch Parameters

☐ Cached Reauthentication
Discovery Period
 seconds
EAPOL Timeout
 seconds
Maximum EAPOL Requests

☐ AOS-S Specific Parameters
☐ AOS-CX Specific Parameters

MAC Parameters

MAC Address Delimiter

MAC Address Case

Max Authentication Failure

☒ Reauthentication
Reauthentication Interval
 seconds
☐ Switch Specific Parameters

Map Dot1X Auth, MAC Auth, and AAA profiles to Device Function(s) and Scope

*** Note:** The dot1xauth & macauth profiles must be mapped before the aaa-profile to ensure correct application of all elements

Valid mappings are:

CORE_SWITCH (CX)	AOSS_CORE_SWITCH
AGG_SWITCH (CX)	AOSS_AGG_SWITCH
ACCESS_SWITCH (CX)	AOSS_ACCESS_SWITCH

In the following three examples, the profiles are mapped to the GLOBAL Scope and the following Device Functions: **Access Switch, Aggregation Switch, Core Switch**

1. Apply Device Function & Scope for dot1xauth profile:

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where `{{name}}` = my_wired_aaa_flood

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "dot1xauth/my_wired_aaa_flood"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "dot1xauth/my_wired_aaa_flood"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "dot1xauth/my_wired_aaa_flood"
    }
  ]
}
```

2. Apply Device Function & Scope for macxauth profile:**POST** `{{baseUrl}}/network-config/v1/scope-maps`Body where `{{name}}` = `my_wired_aaa_flood`

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "macauth/my_wired_aaa_flood"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "macauth/my_wired_aaa_flood"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "macauth/my_wired_aaa_flood"
    }
  ]
}
```

3. Apply Device Function & Scope for aaa-profile:**POST** `{{baseUrl}}/network-config/v1/scope-maps`Body where `{{name}}` = `my_wired_aaa_flood`

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "aaa-profile/my_wired_aaa_flood"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "aaa-profile/my_wired_aaa_flood"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "aaa-profile/my_wired_aaa_flood"
    }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

Name	Assigned Device Function	Assigned Scope
 my_wired_aaa_flood Port Authentication with Flood	Access Switch Core Switch Aggregation Switch	Global

[\(return to TOC\)](#)

SWITCHING

Create VRF

Required: Create API & Scope-Map API

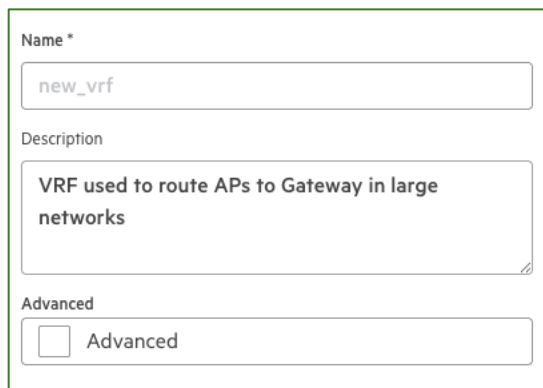
In some cases, it may be necessary to create multiple VRFs to ensure routing remains separate, particularly between management VLANs and VendorNet VLANs where the CX Switch may need to route. In such cases, create a VRF and assign the VLAN interface to the appropriate VRF.

POST `{{baseUrl}}/network-config/v1/vrfs/{{vrf_name}}`

Body where `{{vrf_name}}` = new_vrf

```
{
  "name": "new_vrf",
  "description": "VRF used to route APs to Gateway in large networks"
}
```

The configuration above is based on the following settings:



The screenshot shows a web form for configuring a VRF. It has three sections: 'Name *' with a text input field containing 'new_vrf'; 'Description' with a text area containing 'VRF used to route APs to Gateway in large networks'; and 'Advanced' with a checkbox labeled 'Advanced' that is currently unchecked.

Map VRF to Device Function(s) and Scope

CORE_SWITCH (CX)
 AGG_SWITCH (CX)
 ACCESS_SWITCH (CX)

In the following example, the Admin User is mapped to the GLOBAL Scope and the following Device Functions: **Access Switch, Aggregation Switch, Core Switch**

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where `{{vrf_name}}` = new_vrf

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "vrfs/new_vrf"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "vrfs/new_vrf"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "vrfs/new_vrf"
    }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

Name	Assigned Device Function	Assigned Scope
 new_vrf VRF used to route APs to Gateway in...	Access Switch Aggregation Switch Core Switch	Global

[\(return to TOC\)](#)

Switch Port Profile

Required: Create API & Scope-Map API

Switch Port Profiles provide a way to apply common configuration to many ports through a profile containing the necessary settings. These can be used to set all ports to authenticate, enable loop-protect, STP, etc. Port Profiles can help quickly configure a switch.

To create a Switch Port Profile, two API calls are needed:

- Create Port Profile
- Map the Port Profile to Device Function and Scope

Prior to configuring the Port Profile, other configurations may need to be created, such as VLAN, AAA Authentication Profile, RADIUS Server(s), and RADIUS Group.

POST `{{baseUrl}}/network-config/v1/sw-port-profiles/{{profile_name}}`

Body where `{{profile_name}}` = wlan_ap

```
{
  "profile-name": "wlan_ap",
  "description": "AP Interface Profile",
  "mode": "AUTO",
  "enable": true,
  "routing": false,
  "speed-duplex": "AUTO",
  "mtu": 9198,
  "switchport": {
    "interface-mode": "TRUNK",
    "native-vlan": 101,
    "tag": false,
    "trunk-vlan-ranges": [
      "101",
      "300",
      "1000",
      "1001"
    ]
  },
  "cdp": {
    "enable": true
  },
  "udld": {
    "enable": false
  },
  "qos": {
    "trust": "DSCP"
  },
  "stp": {
    "enable": true,
    "bpdu-filter": false,
    "bpdu-guard": false,
    "loop-guard": false,
  }
}
```



```

    "root-guard": false,
    "rpvst-filter": false,
    "rpvst-guard": false,
    "tcn-guard": false,
    "admin-edge-port": true
  },
  "loop-protect": {
    "enable": true,
    "action": "TX_DISABLE"
  },
  "poe": {
    "enabled": true,
    "allocation-method": "USAGE",
    "priority": "CRITICAL",
    "pre-std-detect": false
  },
  "lldp": {
    "mode": "TX_RX"
  }
}

```

The configuration above is based on the following settings:

General	Network Parameters	Loop Prevention	Fault Monitoring
Name * wlan_ap	☐ Use Switchport Alias	☒ Loop Protection	☐ Enable Fault Monitoring
Description AP Interface Profile	VLAN Mode * ☐ Access ☒ Trunk	VLANs 	QoS QoS Trust DSCP
Device Switch	Native VLAN * 101	Action Transmit disable	Advanced
☒ Admin State	☐ Tag	STP Priority 0 (Highest) <input type="range" value="15"/> 15 (Lowest)	Rate Limit ☐ Broadcast ☐ Multicast ☐ Unknown Unicast ☐ ICMP
Speed/Duplex * Auto	Allowed VLANs * 101,300,1000,1001	STP Cost 	Telemetry IP Client Tracker Auto
MTU 9198	☐ DHCPv4 Snooping Trust	STP Options ☒ Admin Edge ☐ BPDU Guard ☐ BPDU Filter ☐ Root Guard ☐ Loop Guard ☐ TCN Guard ☐ RPVST Filter ☐ RPVST Guard	IP Tracking Client Limit
PoE ☒ Enable PoE	☐ DHCPv6 Snooping Trust	Security ☐ Enable Port Authentication	☐ Enable Application Recognition
Priority Critical	DHCPv6 Snooping Max Binding 	Policy None	
PoE Allocation By Usage	☐ ARP Inspection Trust	☐ Access List	
☐ Pre-Standard Detect			
UDLD ☐ Enable UDLD			
LLDP and CDP LLDP Mode Transmit and Receive			
☒ Enable CDP			
LAG (Early Preview) Configure the ports as LAG members			
☐ Enable LAG			

Map Switch Port Profile to Device Function(s) and Scope

Valid mappings are:

CORE_SWITCH (CX)	AOSS_CORE_SWITCH
AGG_SWITCH (CX)	AOSS_AGG_SWITCH
ACCESS_SWITCH (CX)	AOSS_ACCESS_SWITCH

In the following example, the Switch Port Profile is mapped to the GLOBAL Scope and the following Device Functions: **Access Switch, Aggregation Switch, Core Switch**

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where `{{profile_name}}` = wlan_ap

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "sw-port-profiles/wlan_ap"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "sw-port-profiles/wlan_ap"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "sw-port-profiles/wlan_ap"
    }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

Name	Device Type	Assigned Device Function	Assigned Scope
 wlan_ap AP Interface Profile	Switch	Aggregation Switch Access Switch Core Switch	Global

With the switch Port Profile created, the configuration can be easily applied to a single interface, or multiple interfaces at one time, speeding up the initial configuration.

Example of a single interface Port Profile configuration

The screenshot shows the HPE Aruba Central interface configuration page for a P5-CX6100 switch. The left sidebar shows the library with various site collections, sites, devices, and device groups. The main panel displays the 'Switch Interface Configuration' for the P5-CX6100 switch, showing a table of ports and their configurations. The 'Port Profile' column shows that the 'wlan_ap' profile is applied to port 1/1/2. The right sidebar shows the 'Configure Interface' dialog, where the 'Port' is set to 1/1/2, 'Use Port Profile' is checked, and the 'Port Profile' is set to 'wlan_ap'. A 'Save' button is visible at the bottom right.

Port	Admin Status	Speed/Dup...	Port Profile	LAG	VLAN Mode
1/1/1	Enabled	Auto	-	-	Trunk
1/1/2	Enabled	Auto	-	-	Access
1/1/3	Enabled	Auto	-	-	Access
1/1/4	Enabled	Auto	-	-	Access
1/1/5	Enabled	Auto	-	-	Trunk
1/1/6	Enabled	Auto	-	-	Access
1/1/7	Enabled	Auto	-	-	Access

Example of multiple interfaces Port Profile configuration

The screenshot shows the HPE Aruba Central interface configuration page for a P5-CX6100 switch. The left sidebar shows the library with various site collections, sites, devices, and device groups. The main panel displays the 'Switch Interface Configuration' for the P5-CX6100 switch, showing a table of ports and their configurations. The 'Port Profile' column shows that the 'wlan_ap' profile is applied to ports 1/1/2, 1/1/3, 1/1/4, 1/1/5, and 1/1/6. The right sidebar shows the 'Apply Port Profile' dialog, where the 'Port Profile' is set to 'wlan_ap'. A 'Save' button is visible at the bottom right.

Port	Admin Status	Speed/Dup...	Port Profile	LAG	VLAN Mode
1/1/1	Enabled	Auto	-	-	Trunk
1/1/2	Enabled	Auto	-	-	Access
1/1/3	Enabled	Auto	-	-	Access
1/1/4	Enabled	Auto	-	-	Access
1/1/5	Enabled	Auto	-	-	Trunk
1/1/6	Enabled	Auto	-	-	Access
1/1/7	Enabled	Auto	-	-	Access

Display of multiple ports with Port Profile applied

Port	Admin Status	Speed/Dup...	Port Profile	LAG	VLAN Mode	Routing
☐ 1/1/1	Enabled	Auto	-	-	Trunk	Disabled
☐ 1/1/2	-	-	wlan_ap	-	-	-
☐ 1/1/3	-	-	wlan_ap	-	-	-
☐ 1/1/4	-	-	wlan_ap	-	-	-
☐ 1/1/5	-	-	wlan_ap	-	-	-
☐ 1/1/6	-	-	wlan_ap	-	-	-
☐ 1/1/7	Enabled	Auto	-	-	Access	Disabled

[\(return to TOC\)](#)

Configure Interface Settings

Required: Modify/Replace API at Device Scope

Configuring a Switch interface can be accomplished by specifying the individual settings or applying an Interface Port Profile, demonstrated in the previous section.

When applying settings, the scope-id is set to the individual device.

Obtain the device scopeld information which will be used to make the configuration changes via API

GET `{{baseUrl}}/network-config/v1/devices`

If either the name of the device or serial number is known, the request can be filtered using the device-name (hostname) or scopeName (serial number) to limit the response output

GET `{{baseUrl}}/network-config/v1/devices?filter=name eq '{{device-name}}'` Where **filter** = `?filter=name eq 'P5-CX6100'`

Response:

```
{
  "offset": null,
  "total": 1,
  "items": [
    {
      "model": "6100 12G CL4 2SFP+ 139W Swch (JL679A)",
      "scopeId": "25765472801",
      "id": "25765472801",
      "scopeName": "TW22KKW56V",
      "deployment": "Standalone",
      "deviceType": "SWITCH",
      "name": "P5-CX6100",
      "type": "network-config/devices",
      "persona": null,
      "deviceGroupName": "CNX",
      "siteId": "25964326768",
      "deviceGroupId": "834897312",
      "siteName": "P5-DCARW",
      "mac": "38:21:c7:ea:78:90",
      "ipv4": "10.4.3.130"
    }
  ],
  "count": 1,
  "response": {
    "status": "SUCCESS",
    "message": "success",
    "code": "SCOPE-DEVICE-SUC-001"
  }
}
```

To view the Ethernet Interface settings, use the following:

GET `{{baseUrl}}/network-config/v1/ethernet-interfaces?view-type=LOCAL&scope-id={{device_scope_id}}`

The output below is shortened to display a few interfaces that have different configurations

1/1/1	Individual configured with Trunk for ACCESS POINT
1/1/3	Port Profile configured (399-quarantine)
1/1/6	Access interface with default config
1/1/14	Individual configured with Trunk for Uplink

```
{
  "interface": [
    {
      "name": "1/1/1",
      "description": "Access Point",
      "poe": {
        "enabled": true,
        "allocation-method": "USAGE",
        "priority": "HIGH",
        "pre-std-detect": false
      },
      "lldp": {
        "mode": "TX_RX"
      },
      "enable": true,
      "routing": false,
      "speed-duplex": "AUTO",
      "mtu": 1500,
      "dpi-enable": false,
      "switchport": {
        "interface-mode": "TRUNK",
        "native-vlan": 101,
        "tag": false,
        "trunk-vlan-all": false,
        "trunk-vlan-ranges": [
          "101",
          "1000"
        ]
      },
      "cdp": {
        "enable": true
      },
      "udld": {
        "enable": false
      },
      "stp": {
        "bpdu-filter": false,
        "bpdu-guard": false,
        "loop-guard": false,
        "root-guard": false,
        "rpvst-filter": false,
        "rpvst-guard": false,
        "tcn-guard": false,

```

```

        "admin-edge-port": false,
        "priority": 8
    },
    "loop-protect": {
        "enable": false
    },
    "dhcpv4-snooping": {
        "trust": false
    },
    "dhcpv6-snooping": {
        "trust": false
    },
    "ip-source-lockdown": {
        "ipv4": false,
        "ipv6": false
    },
    "mode": "AUTO",
    "dynamic-arp-inspection": {
        "trust": false
    },
    "aaa": {
        "authentication": {
            "concurrent-onboarding": false,
            "device-profile-secure": false
        }
    }
},
{
    "name": "1/1/3",
    "description": "quarantine port",
    "sw-profile": "399-quarantine",
    "poe": {
        "enabled": true,
        "allocation-method": "USAGE",
        "priority": "LOW",
        "pre-std-detect": false
    },
    "lldp": {
        "mode": "TX_RX"
    },
    "enable": true,
    "routing": false,
    "speed-duplex": "AUTO",
    "mtu": 1500,
    "dpi-enable": true,
    "switchport": {
        "interface-mode": "ACCESS",
        "access-vlan": 399
    },
    "cdp": {
        "enable": true
    },
    "udld": {
        "enable": false
    },
    "qos": {
        "trust": "NONE"
    },
    "stp": {
        "enable": true,

```

```

        "bpdu-filter": false,
        "bpdu-guard": false,
        "loop-guard": false,
        "root-guard": false,
        "rpvst-filter": false,
        "rpvst-guard": false,
        "tcn-guard": false,
        "admin-edge-port": true
    },
    "loop-protect": {
        "enable": true,
        "action": "TX_DISABLE",
        "vlans": [
            399
        ]
    },
    "mode": "AUTO"
},
{
    "name": "1/1/6",
    "poe": {
        "enabled": true,
        "allocation-method": "USAGE",
        "priority": "LOW",
        "pre-std-detect": false
    },
    "enable": true,
    "speed-duplex": "AUTO",
    "mtu": 1500,
    "switchport": {
        "interface-mode": "ACCESS",
        "access-vlan": 1
    }
},
{
    "name": "1/1/14",
    "poe": {
        "enabled": false,
        "pre-std-detect": false
    },
    "lldp": {
        "mode": "TX_RX"
    },
    "enable": true,
    "routing": false,
    "speed-duplex": "AUTO",
    "mtu": 1500,
    "dpi-enable": false,
    "switchport": {
        "interface-mode": "TRUNK",
        "native-vlan": 100,
        "tag": false,
        "trunk-vlan-all": true
    },
    "cdp": {
        "enable": true
    },
    "udld": {
        "enable": false
    }
},

```



```

    "stp": {
      "bpdu-filter": false,
      "bpdu-guard": false,
      "loop-guard": false,
      "root-guard": false,
      "rpvst-filter": false,
      "rpvst-guard": false,
      "tcn-guard": false,
      "admin-edge-port": false,
      "priority": 8
    },
    "loop-protect": {
      "enable": false
    },
    "dhcpv4-snooping": {
      "trust": false
    },
    "dhcpv6-snooping": {
      "trust": false
    },
    "ip-source-lockdown": {
      "ipv4": false,
      "ipv6": false
    },
    "mode": "AUTO",
    "dynamic-arp-inspection": {
      "trust": false
    },
    "aaa": {
      "authentication": {
        "concurrent-onboarding": false,
        "allow-flood-traffic": false,
        "bpdu-auth": {
          "lldp": false,
          "cdp": false
        },
        "bpdu-bypass": {
          "lldp": false,
          "cdp": false
        },
        "bpdu-proxy-logout": {
          "cdp": false
        }
      },
      "authorization": {
        "radius-override-enable": false
      },
      "security-violation": {
        "shutdown-recovery-enable": false
      }
    }
  },
}

```

To update individual interface (non port-profile) configuration use **PATCH** to make the necessary changes.

PATCH `{{baseUrl}}/network-config/v1/ethernet-interfaces/{{profile-name}}?object-type=LOCAL&scope-id={{device_scope_id}}&device-function={{function}}`

Where `{{function}}` = ACCESS_SWITCH and `{{profile-name}}` = 1%2F1%2F6 (1/1/6)
`{{device_scope_id}}` = the scope-id of the device

```
{
  "name": "1/1/6",
  "description": "Doorlock Server",
  "switchport": {
    "interface-mode": "ACCESS",
    "access-vlan": 300
  }
}
```

Before

```
{
  "name": "1/1/6",
  "retain-profile-configs": false,
  "poe": {
    "enabled": true,
    "allocation-method": "USAGE",
    "priority": "LOW",
    "pre-std-detect": false
  },
  "enable": true,
  "speed-duplex": "AUTO",
  "mtu": 1500,
  "switchport": {
    "interface-mode": "ACCESS",
    "access-vlan": 1
  }
}
```

After

```
{
  "name": "1/1/6",
  "description": "Doorlock Server",
  "poe": {
    "enabled": true,
    "allocation-method": "USAGE",
    "priority": "LOW",
    "pre-std-detect": false
  },
  "enable": true,
  "routing": false,
  "speed-duplex": "AUTO",
  "mtu": 1500,
  "switchport": {
    "interface-mode": "ACCESS",
    "access-vlan": 300
  }
}
```

To change an interface with a **Port Profile**, use **PUT** to change the profile

PUT `{{baseUrl}}/network-config/v1/ethernet-interfaces/{{profile-name}}?object-type=LOCAL&scope-id={{device_scope_id}}&device-function={{function}}`

Where `{{function}}` = ACCESS_SWITCH and `{{profile-name}}` = 1%2F1%2F3 (1/1/3)
`{{device_scope_id}}` = the scope-id of the device

```
{
  "name": "1/1/3",
  "sw-profile": "auth-port-profile"
}
```

The results are on the following pages:

Before:

```

{
  "name": "1/1/3",
  "description": "quarantine port",
  "sw-profile": "399-quarantine",
  "poe": {
    "enabled": true,
    "allocation-method": "USAGE",
    "priority": "LOW",
    "pre-std-detect": false
  },
  "lldp": {
    "mode": "TX_RX"
  },
  "enable": true,
  "routing": false,
  "speed-duplex": "AUTO",
  "mtu": 1500,
  "dpi-enable": true,
  "switchport": {
    "interface-mode": "ACCESS",
    "access-vlan": 399
  },
  "cdp": {
    "enable": true
  },
  "udld": {
    "enable": false
  },
  "qos": {
    "trust": "NONE"
  },
  "stp": {
    "enable": true,
    "bpdu-filter": false,
    "bpdu-guard": false,
    "loop-guard": false,
    "root-guard": false,
    "rpvst-filter": false,
    "rpvst-guard": false,
    "tcn-guard": false,
    "admin-edge-port": true
  },
  "loop-protect": {
    "enable": true,
    "action": "TX_DISABLE",
    "vlangs": [
      399
    ]
  },
  "mode": "AUTO"
}

```

After:

```

{
  "name": "1/1/3",
  "sw-profile": "auth-port-profile",
  "poe": {
    "enabled": true,
    "allocation-method": "USAGE",
    "priority": "LOW",
    "pre-std-detect": false
  },
  "lldp": {
    "mode": "TX_RX"
  },
  "enable": true,
  "routing": false,
  "speed-duplex": "AUTO",
  "mtu": 1500,
  "dpi-enable": true,
  "switchport": {
    "interface-mode": "ACCESS",
    "access-vlan": 1000
  },
  "cdp": {
    "enable": true
  },
  "udld": {
    "enable": false
  },
  "qos": {
    "trust": "NONE"
  },
  "stp": {
    "enable": true,
    "bpdu-filter": false,
    "bpdu-guard": false,
    "loop-guard": false,
    "root-guard": false,
    "rpvst-filter": false,
    "rpvst-guard": false,
    "tcn-guard": false,
    "admin-edge-port": true
  },
  "loop-protect": {
    "enable": true,
    "action": "TX_DISABLE",
    "vlans": [
      1000
    ]
  },
  "mode": "AUTO",
  "aaa": {
    "authentication": {
      "dot1x-auth": {
        "enable": true,
        "eapol-max-requests": 1,
        "eapol-timeout": 5,

```

```

        "reauth-enable": true,
        "reauth-period": 28800,
        "cached-reauth-enable": false
    },
    "mac-auth": {
        "enable": true,
        "reauth-enable": true,
        "reauth-period": 28800,
        "cached-reauth-enable": false
    },
    "dot1xauth-server-group": "RADIUS-1-Group",
    "macauth-server-group": "RADIUS-1-Group",
    "onboarding-precedence": [
        "DEVICE_PROFILE",
        "AAA"
    ],
    "concurrent-onboarding": true,
    "cloud-auth": false,
    "auth-mode": "CLIENT_MODE",
    "allow-flood-traffic": false,
    "client-limit": 2,
    "bpdu-auth": {
        "lldp": true,
        "cdp": true
    },
    "bpdu-bypass": {
        "lldp": true,
        "cdp": true
    },
    "bpdu-proxy-logout": {
        "cdp": false
    },
    "device-profile-secure": false
},
"authorization": {
    "radius-override-enable": false
},
"security-violation": {
    "shutdown-recovery-enable": false
}
}

```

[\(return to TOC\)](#)

LAG (Port Channel)

Required: Create API at Device Scope

Link Aggregation allows multiple interfaces to be combined to create a larger-bandwidth interface with fault-tolerance.

POST `{{baseUrl}}/network-config/v1/portchannels/{{id}}?object-type=LOCAL&scope={{scope-id}}&device_function={{function}}`

Where:

`{{id}}` = 101
`{{scope-id}}` = switch scope-id
`{{function}}` = function (CORE_SWITCH, AGG_SWITCH, ACCESS_SWITCH)

```
{
  "name": "101",
  "port-list": [
    "1/1/11",
    "1/1/12"
  ],
  "enable": true,
  "routing": false,
  "stp": {
    "bpdu-filter": false,
    "bpdu-guard": false,
    "loop-guard": false,
    "root-guard": false,
    "rpvst-filter": false,
    "rpvst-guard": false,
    "tcn-guard": false,
    "admin-edge-port": false,
    "priority": 4
  },
  "loop-protect": {
    "enable": false
  },
  "aaa-lag": {
    "authentication-lag": {
      "allow-flood-traffic": false,
      "bpdu-auth": {
        "lldp": true,
        "cdp": true,
        "mac-type": "CHASSIS_MAC"
      },
      "bpdu-bypass": {
        "lldp": false,
        "cdp": false
      },
      "bpdu-proxy-logoff": {
        "cdp": false
      }
    },
    "device-profile-secure": false
  }
}
```

```

    },
    "security-violation": {
        "shutdown-recovery-enable": false
    }
},
"lACP": {
    "mode": "ACTIVE",
    "fallback-static": false,
    "rate": "SLOW"
},
"switchport": {
    "interface-mode": "TRUNK",
    "native-vlan": 100,
    "tag": false,
    "trunk-vlan-all": false,
    "trunk-vlan-ranges": [
        "100",
        "1000"
    ]
},
"dhcpv4-snooping": {
    "trust": false
},
"dhcpv6-snooping": {
    "trust": false
},
"ip-source-lockdown": {
    "ipv4": false,
    "ipv6": false
},
"mode": "AUTO",
"dynamic-arp-inspection": {
    "trust": false
},
"trunk-type": "LACP"
}

```

The API settings above result in the following settings

Name	LAG Type	LACP	Ports	Enable	VLAN Mode	VLAN	Routing
101	LAG	Yes	1/1/11, 1/1/12	Yes	Trunk	Allowed: 100,1000 Native: ... Allowed: ...	No ...

Alternatively: to allow All VLANs,

"trunk-vlan-all": **true**

LAG

Configure Link Aggregation (LAG) for ports.



Set IP Tracker to 'Auto' when creating a LAG for use as an Inter-Switch Link (ISL) between VSX peers.

General



Ports

1/1/11

1/1/12

LAG ID *

1-480

101

Description



Use Port Profile



Enable

LAG Type *



LAG



MCLAG



LACP

LACP Mode *

Active



Rate *

Slow (Default)



LACP Fallback Static

Network Parameters



Routing

VLAN Mode *



Access



Trunk

Native VLAN *

100



Tag

Allowed VLANs *

100,1000



DHCPv4 Snooping Trust

DHCPv4 Snooping Max Binding



DHCPv6 Snooping Trust

DHCPv6 Snooping Max Binding



ARP Inspection Trust



Block Port Until Device Profile Active

Policies

Policy

None



Access List

QoS

QoS Trust

Select



Specify CoS Value

0-7

Specify DSCP Value

0-63



Shape Outbound Traffic

Loop Prevention

☐ Loop Protection

STP Port-Priority

0 (Highest)

15 (Lowest)



STP Port-Cost

STP Options

- ☐ BPDU Filter
- ☐ BPDU Guard
- ☐ Admin Edge
- ☐ Root Guard
- ☐ Loop Guard
- ☐ TCN Guard
- ☐ RPVST Filter
- ☐ RPVST Guard

Rate limit

☐ Broadcast

☐ Multicast

☐ Unknown Unicast

☐ ICMP

Security

☐ IPv4 Source Lockdown

☐ IPv6 Source Lockdown

Authentication

Client Limit

☐ Allow Flood Traffic

Authentication Mode

Select



LLDP Support

☒ Allow LLDP Authentication

☒ Chassis MAC

☐ Interface MAC

☐ Allow LLDP Bypass

CDP Support

☒ Allow CDP Authentication

☐ Allow CDP Bypass

☐ Allow CDP Proxy-logout

Security Violation

Action

Select



Telemetry

IP Client Tracker

Auto



IP Tracking Client Limit

[\(return to TOC\)](#)

Switch System Profile

Required: Create API & Scope-Map API

A Switch System Profile defines the timezone, loop-protect timer, enables AAA for the switch, defines the default RADIUS Group, and port-security for the switch.

NOTE: Only one Switch System Profile can be assigned to a device

POST `{{baseUrl}}/network-config/v1/switch-system/{{profile_name}}`

Where:

`{{profile_name}}` = my-cx-switch-profile-EST

`{{radius_group}}` = RADICAL_Server_Group

```
{
  "name": "my-cx-switch-profile-EST",
  "description": "Switch profile for all CX switches",
  "timezone": {
    "timezone-name": "US/Eastern",
    "hour-offset": -4,
    "minute-offset": 0
  },
  "usb-port-disable": false,
  "enable-unsupported-transceiver": true,
  "access-list": {
    "log-enable": true,
    "log-timer": 300
  },
  "loop-protect": {
    "transmit-interval": 5,
    "re-enable-timer": 180,
    "trap": true
  },
  "aaa": {
    "dot1x-auth": {
      "enable": true,
      "radius-server-group": "RADICAL_Server_Group"
    },
    "mac-auth": {
      "enable": true,
      "radius-server-group": "RADICAL_Server_Group"
    },
    "cached-critical-role": {
      "enable": false,
      "persistent-storage-enable": false
    }
  },
  "port-security": {
    "enable": true
  }
}
```

The configuration above is based on the following settings:

PropertiesReferences

Switch System

Manage switch system configuration.

General

Name *

Description

Location

Contact

Timezone *

Virtual MAC Address

ⓘ Must be configured at the device level.

☐ Disable USB-Port

☒ Enable Unsupported Transceiver

☒ Enable Port Security

☐ AOS-S Specific Parameters

AAA

☒ Enable 802.1X

☒ Enable MAC Authentication

MAC Address Format

MAC Radius Auth Method

802.1X Authentication Server Group *

MAC Authentication Server Group *

☐ AOS-S Specific Parameters

Auth Survivability

☐ Enable Cached Critical Role

☐ Persistent Storage Enable

Cache Timeout

Loop Protect

Re-Enable Timer

 seconds

Transmit Interval

1 Second

5 Seconds

10 Seconds

☒ Trap on Loop Detection

AOS-S Specific Parameters

Access List

☒ Log Enabled

Log Timer

Power

Configuration supported for CX switches.

Power Redundancy

Stack Power Redundancy VSF Member

Consumption Average Period

 seconds

System Internal VLAN Range

(Only for 6200)

Telemetry

Configure recommended telemetry for CX switches.

☐ Enable Device Fingerprinting Profile

☐ Enable IP Client Tracker

ICMP

☒ Enable ICMP Redirect

Map Switch System Profile to Device Function(s) and Scope

Valid mappings are:

CORE_SWITCH (CX)	AOSS_CORE_SWITCH
AGG_SWITCH (CX)	AOSS_AGG_SWITCH
ACCESS_SWITCH (CX)	AOSS_ACCESS_SWITCH

In the following example, the Auth Server Group is mapped to the GLOBAL Scope and the following Device Functions: **Access Switch, Aggregation Switch, Core Switch**

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where `{{profile_name}}` = my-cx-switch-profile-EST

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CORE_SWITCH",
      "resource": "switch-system/my-cx-switch-profile-EST"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "AGG_SWITCH",
      "resource": "switch-system/my-cx-switch-profile-EST"
    },
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "ACCESS_SWITCH",
      "resource": "switch-system/my-cx-switch-profile-EST"
    }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

Profiles	Roles & Policies	Named Objects	Services
Library > System > Switch System			
Q Search			Create Profile
5 items			
Name	Assigned Device Function	Assigned Scope	
my-cx-switch-profile-EST Switch profile for all CX switches	Access Switch Core Switch Aggregation Switch	Global	

[\(return to TOC\)](#)

WIRELESS

To review configuration options for WLAN creation, view the [HPE Aruba Developer site](#)

In many cases it may be desirable to deploy configuration elements (such as WLANs) only to specific areas. With New Central, customers and partners can create Device Groups and place devices into those groups. Inheritance flows to the device in the following order:

Global < Site Collections < Sites < Device Groups < Device Level

See the section on [New Central Hierarchy](#) for further information regarding Inheritance Precedence.

The determination of where to deploy the WLAN could be to provide specific Device Groups the desired WLAN service. Deploy the WLAN to the scopes where the services should be deployed.

When creating a WLAN, a default role is automatically created with the same name as the ESSID and will be assigned to clients that connect to the network unless another role assignment is made (RADIUS Response, Assignment Rules, etc.)

The below image shows an example of the scope where each WLAN could be assigned and where the inheritance flows. They are created in the Library then assigned to the appropriate scope-id and device function (CAMPUS_AP).



- WLAN: **MARSHA_W1**
Scope Assigned: Global
Where Available: All Access Points inherit
- WLAN: **MARSHA_WP**
Scope Assigned: Global
Where Available: All Access Points inherit
- WLAN: **MarriottBonvoy_Guest**
Scope Assigned: {Device Group Guest Rooms} | (Device Group Public)
Where Available: Only devices in the specified Device Groups inherit
- WLAN: **MarriottBonvoy_Conference**
Scope Assigned: (Device Group Conf1 | Device Group Conf n)
Where Available: Only devices in the specified Device Groups inherit
- WLAN: **Vanity_Conf**
Scope Assigned: (Device Group Conf1 | Device Group Conf n)
Where Available: Only devices in the specified Device Groups inherit

Underlay WLAN

Required: Create API & Scope-Map API

Underlay WLANs deliver the client traffic to the switchport to which the AP is connected. Best practice is placing wireless client traffic on different VLAN than AP management.

To create an Underlay WLAN, two API calls are needed:

- Create WLAN (underlay)
- Map the WLAN to Device Function and Scope(s)

NOTE * If the WLAN includes spaces, replace each space with `%20` in the **POST** URL address. Leave the space(s) in the body ssid & essid fields:

POST URL	POST .../wlan-ssids/Vanity%20Group
Body	"ssid": "Vanity Group" "essid": {"name": "Vanity Group"}

POST `{{baseUrl}}/network-config/v1/wlan-ssids/{{essid_name}}`

Body where `{{essid_name}}` = test_ssid

```
{
  "ssid": "test_ssid",
  "enable": true,
  "forward-mode": "FORWARD_MODE_BRIDGE",
  "dmo": {
    "enable": true,
    "channel-utilization-threshold": 90,
    "clients-threshold": 6
  },
  "broadcast-filter-ipv4": "BCAST_FILTER_ARP",
  "broadcast-filter-ipv6": "UCAST_FILTER_RA",
  "optimize-mcast-rate": false,
  "ssid-utf8": true,
  "essid": {
    "use-alias": false,
    "name": "test_ssid"
  },
  "advertise-apname": false,
  "disable-on-6ghz-mesh": false,
  "dot11k": true,
  "dtim-period": 1,
  "ftm-responder": false,
  "hide-ssid": false,
  "auth-req-thresh": 0,
  "explicit-ageout-client": false,
  "inactivity-timeout": 1000,
  "local-probe-req-thresh": 0,
  "max-clients-threshold": 1024,
  "rf-band": "24GHZ_5GHZ",
  "rrm-quiet-ie": false,
  "high-throughput": {
    "enable": true,
    "very-high-throughput": true
  }
},
```



```

    "g-legacy-rates": {
      "basic-rates": [
        "RATE_12MB",
        "RATE_24MB"
      ],
      "tx-rates": [
        "RATE_12MB",
        "RATE_18MB",
        "RATE_24MB",
        "RATE_36MB",
        "RATE_48MB",
        "RATE_54MB"
      ]
    },
    "a-legacy-rates": {
      "basic-rates": [
        "RATE_12MB",
        "RATE_24MB"
      ],
      "tx-rates": [
        "RATE_12MB",
        "RATE_18MB",
        "RATE_24MB",
        "RATE_36MB",
        "RATE_48MB",
        "RATE_54MB"
      ]
    },
    "high-efficiency": {
      "enable": true
    },
    "extremely-high-throughput": {
      "enable": true,
      "mlo": false
    },
    "opmode": "ENHANCED_OPEN",
    "use-ip-for-calling-station-id": false,
    "called-station-id": {
      "type": "MAC_ADDRESS",
      "include-ssid": false
    },
    "cloud-auth": false,
    "wpa3-transition-mode-enable": true,
    "denylist": true,
    "max-authentication-failures": 0,
    "enforce-dhcp": false,
    "pan": false,
    "vlan-selector": "VLAN_RANGES",
    "vlan-id-range": [
      "1000"
    ],
    "out-of-service": "NONE",
    "client-isolation": false
  }
}

```

The configuration above is based on the following settings:

Update WLAN Profile

General

Name *

Description

Type
Access

☐ Use Alias

ESSID Name *

☒ 2.4 GHz
☒ 5 GHz
☐ 6 GHz

☐ Disable Network

Network Configuration ⓘ
☐ Most Compatible
☐ Balanced
☐ High Density
☒ Custom

VLAN

Traffic Forwarding Mode *
☒ Bridge
☐ Tunnel
☐ Mixed

☐ Use Named VLAN

Default VLAN *

Advanced

Options
☒ Broadcast/Multicast
☒ Data Rates
☒ Bandwidth Control
☒ Wi-Fi Multimedia
☒ Miscellaneous
☒ Time Range

Broadcast/Multicast
Broadcast Filtering ⓘ
ARP
☒ IPv6 RA and ND Optimization
☐ IPv6 ND Proxy
☐ Multicast Transmission Optimization
☒ Dynamic Multicast Optimization (DMO)

DHDP channel utilization threshold

DHDP client threshold

Data Rates
2.4 GHz Basic Rates
2 rates selected
Enabled Basic Rates
12, 18
5 GHz Basic Rates
2 rates selected
Enabled Basic Rates
12, 24

Bandwidth Control
Wi-Fi Multimedia
☒ Wi-Fi 4
☒ Wi-Fi 5
☒ Wi-Fi 6
☒ Wi-Fi 7

Security

Security Level *
☐ Enterprise
☐ Personal
☒ Open

Key Management *
Enhanced Open

Captive Portal
Type
None

Authentication
☐ MAC Authentication

Advanced
☒ Denylisting
☐ Client Isolation
☐ Enforce DHCP
☒ WPA3 Transition
Max Authentication Failures
0
☐ Use IP for Calling Station ID
Called Station ID Type
MAC Address
☐ Called Station ID Include SSID
Fast Roaming
☒ 802.11k
☐ BSSW Quiet IE

Access

Default Role -
☐ Override default role
ⓘ Default policies for roles have been created under Roles & Policies
> Role-Based policies? Please finish the configuration by editing said policies.

Advanced

Map WLAN to Device Function(s) and Scope

Valid mappings are:

CAMPUS_AP

In the following example, the WLAN is mapped to the GLOBAL Scope and the following Device Functions: **CAMPUS_AP**

NOTE * If the WLAN includes spaces, replace each space with %20 in the resource field

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where `{{ssid_name}}` = test_ssid:

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CAMPUS_AP",
      "resource": "wlan-ssids/test_ssid"
    }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

1 item			
Name	Type	Assigned Device Function	Assigned Device Scope
 test_ssid	Access	Campus Access Point	Global

To map the WLAN to a specific Device Group(s), modify the scope-name & scope-id and POST to each group as desired.

```
{
  "scope-map": [
    {
      "scope-name": "{{device_group_scope-id}}",
      "scope-id": {{device_group_scope-id}},
      "persona": "CAMPUS_AP",
      "resource": "wlan-ssids/test_ssid"
    }
  ]
}
```

The API settings above result in the following Device Function and scope assignment

Name	Type	Assigned Device Function	Assigned Device Scope	Status
 test_ssid	Access	Campus Access Point	848811599	Enabled

[\(return to TOC\)](#)

Overlay WLAN

Required: Create API & Scope-Map API

Creating an Overlay (tunneled) WLAN requires a few more steps than Underlay WLANs. The process includes the following:

1. Create WLAN Profile
2. Create Overlay Profile
3. Map WLAN Profile to Persona=CAMPUS_AP & Scope(s) [global or Device Groups*]
4. Map Role to Persona=MOBILITY_GW & Scope
5. Map Overlay Profile to Persona=CAMPUS_AP & Scope(s) [global or Device Groups*]

* At the time of this writing, Overlay WLANs should be created in the Library and mapped to Device Group Scope(s). Mapping to Global Scope will be available in the future.

NOTE * If the WLAN includes spaces, replace each space with `%20` in the **POST** URL address
 Leave the space(s) in the body ssid & essid fields:

POST URL	POST .../wlan-ssids/Vanity%20Group
Body	"ssid": "Vanity Group" "essid": {"name": "Vanity Group"}

Steps to configure Overlay WLAN

1. Create WLAN Profile

Required: Create API & Scope-Map API

POST `{{baseUrl}}/network-config/v1/wlan-ssids/{{essid_name}}`

Body where `{{essid_name}}` = MAR_WP

Ensure the “forward-mode” is `FORWARD_MODE_L2`

The role will be auto-created with the name of the ESSID.

```
{
  "ssid": "MAR_WP",
  "enable": true,
  "forward-mode": "FORWARD_MODE_L2",
  "cluster-preemption": false,
  "dmo": {
    "enable": false,
    "channel-utilization-threshold": 90,
    "clients-threshold": 6
  },
  "broadcast-filter-ipv4": "BCAST_FILTER_ARP",
  "broadcast-filter-ipv6": "UCAST_FILTER_RA",
  "local-proxy-ns": false,
  "optimize-mcast-rate": false,
  "ssid-utf8": true,
  "essid": {
```

```

    "name": "MAR_WP"
  },
  "advertise-apname": false,
  "disable-on-6ghz-mesh": false,
  "dot11k": true,
  "dot11r": false,
  "dtim-period": 1,
  "ftm-responder": false,
  "hide-ssid": false,
  "auth-req-thresh": 0,
  "explicit-ageout-client": false,
  "inactivity-timeout": 1000,
  "local-probe-req-thresh": 0,
  "max-clients-threshold": 1024,
  "rf-band": "BAND_ALL",
  "rrm-quiet-ie": false,
  "high-throughput": {
    "enable": true,
    "very-high-throughput": true
  },
  "g-legacy-rates": {
    "basic-rates": [
      "RATE_12MB",
      "RATE_24MB"
    ],
    "tx-rates": [
      "RATE_12MB",
      "RATE_18MB",
      "RATE_24MB",
      "RATE_36MB",
      "RATE_48MB",
      "RATE_54MB"
    ]
  },
  "a-legacy-rates": {
    "basic-rates": [
      "RATE_12MB",
      "RATE_24MB"
    ],
    "tx-rates": [
      "RATE_12MB",
      "RATE_18MB",
      "RATE_24MB",
      "RATE_36MB",
      "RATE_48MB",
      "RATE_54MB"
    ]
  },
  "high-efficiency": {
    "enable": true
  },
  "extremely-high-throughput": {
    "enable": true,
    "mlo": false,
    "beacon-protection": false
  },
  "wmm-cfg": {

```

```

    "uapsd": false
  },
  "advertise-timing": false,
  "opmode": "WPA3_SAE",
  "mac-authentication": false,
  "personal-security": {
    "passphrase-format": "STRING",
    "wpa-passphrase": "password123"
  },
  "type": "EMPLOYEE",
  "use-ip-for-calling-station-id": false,
  "gw-auth-server": "default",
  "called-station-id": {
    "type": "MAC_ADDRESS",
    "include-ssid": false
  },
  "cloud-auth": false,
  "default-role": "MAR_WP",
  "wpa3-transition-mode-enable": true,
  "denylist": true,
  "max-authentication-failures": 0,
  "enforce-dhcp": false,
  "pan": false,
  "vlan-selector": "VLAN_RANGES",
  "vlan-id-range": [
    "1017"
  ],
  "out-of-service": "TUNNEL_DOWN",
  "client-isolation": false
}

```

2. Create Overlay WLAN Profile **Required**: Create API & Scope-Map API

The cluster name and its scope-id must first be obtained prior to the creation of Overlay WLANs so that the overlay-wlan profile can be properly configured. The [High Availability > Gateway Clustering](#) is only available at the Site or Device Group contexts.

To get the cluster scope-id, execute API for scope-maps and locate the gatewayclusters bearing the name. It will include the scope-id.

GET `{{baseUrl}}/network-config/v1/scope-maps`

Response: *Response data truncated for space...*

```

{
  "scope-name": "25967476218",
  "scope-id": 25967476218,
  "persona": "MOBILITY_GW",
  "resource": "gateway-clusters/P5-site-cluster"
}, Remaining response removed for space

```

Use the cluster-name and scope-id to create the overlay-wlan profile

POST `{{baseUrl}}/network-config/v1/overlay-wlan/{{essid_name}}`

Body where `{{essid_name}}` = MAR_WP

```
{
  "profile": "MAR_WP",
  "overlay-profile-type": "WIRELESS_PROFILE",
  "essid-name": "overlay_PMP",
  "gw-cluster-list": [
    {
      "cluster-redundancy-type": "PRIMARY",
      "cluster": "P5-site-cluster",
      "cluster-scope-id": "25967476218",
      "cluster-type": "CLUSTER_ID",
      "tunnel-type": "GRE"
    }
  ]
}
```

3. Map WLAN Profile to Persona CAMPUS_AP & Scope

Required: Create API & Scope-Map API

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where `{{essid_name}}` = test_ssid:

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CAMPUS_AP",
      "resource": "wlan-ssids/MAR_WP"
    }
  ]
}
```

4. Map Role to Persona MOBILITY_GW and Scope

Required: Create API & Scope-Map API

In this example the scope will be Global

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where `{{role_name}}` = MAR_WP:

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "MOBILITY_GW",
      "resource": "roles/MAR_WP"
    }
  ]
}
```

5. Map Overlay Profile to Persona CAMPUS_AP and Scope

Required: Create API & Scope-Map API

In this example the scope will be Global

POST `{{baseUrl}}/network-config/v1/scope-maps`

Body where `{{essid_name}}` = MAR_WP:

```
{
  "scope-map": [
    {
      "scope-name": "{{global_scope-id}}",
      "scope-id": {{global_scope-id}},
      "persona": "CAMPUS_AP",
      "resource": "overlay-wlan/MAR_WP"
    }
  ]
}
```

[\(return to TOC\)](#)

Update WLANs

Modifying a WLAN involves issuing a PATCH or PUT call via API

- **PATCH** updates the configuration with the modified or additional settings
 - **PUT** replaces the configuration with the data being pushed
- Ensure that all the necessary configuration elements are included or any existing settings not referenced in the PUT API body will be removed

PATCH `{{baseUrl}}/network-config/v1/wlan-ssids/{{ssid_name}}`

NOTE * If the WLAN includes spaces, replace each space with `%20` in the **PATCH/PUT** URL address

Leave the space(s) in the body ssid & essid fields:

PATCH URL **PATCH** `.../wlan-ssids/Vanity%20Group`

Body `"ssid": "Vanity Group"`

One common update use case is to enable/disable the WLAN.
Below shows an example.

PATCH `{{baseUrl}}/network-config/v1/wlan-ssids/Special%20Conference`

Body where SSID = Special Conference

```
{
  "ssid": "Special Conference",
  "enable": false
}
```

Result:

```
{
  "debugId": "7ddb23262a2bac95eeabfff01557f236",
  "errorCode": "SUCC_001",
  "httpStatusCode": 200,
  "message": "success"
}
```

[\(return to TOC\)](#)

Delete WLANs

When deleting the WLAN, multiple steps may need to be taken depending on whether the traffic delivery is Underlay or Overlay. The deletion of the WLAN **DOES NOT** automatically remove the default role created for the WLAN (same name as the WLAN profile).

NOTE * If the WLAN includes spaces, replace each space with **%20** in the **PATCH/PUT** URL address

Leave the space(s) in the body ssid & essid fields:

PATCH URL **PATCH** .../wlan-ssids/**Vanity%20Group**

Body "ssid": "Vanity Group"

Underlay WLAN

When configuring an Underlay WLAN, three elements are created

Persona: CAMPUS_AP "wlan-ssids/{{ssid_name}}"

Persona: CAMPUS_AP "roles/{{ssid_name}}"

Persona: CAMPUS_AP "role-gpids/{{ssid_name}}"

Deleting an Underlay WLAN requires only one step:

1. Delete Underlay WLAN SSID

API call to delete an Underlay WLAN:

DELETE {{baseUrl}}/network-config/v1/wlan-ssids/**Special%20Conference**

ONE PROBLEM CURRENTLY REMAINS

When deleting the WLAN and Role, the Policy sys_allow_all does NOT remove the policy assignment of the role. When deleting via GUI, it does.

This is being addressed by Engineering and should be resolved in such a way that the deletion of the WLAN also removes the rule assignment within the sys_allow_all policy

Overlay WLAN

When creating an Overlay WLAN, seven elements are created

Persona: CAMPUS_AP	"wlan-ssids/{{ssid_name}}"
Persona: CAMPUS_AP	"overlay-wlan/{{ssid_name}}"
Persona: CAMPUS_AP	"roles/{{ssid_name}}"
Persona: MOBILITY_GW	"roles/{{ssid_name}}"
Persona: CAMPUS_AP	"role-gpids/{{ssid_name}}"
Persona: MOBILITY_GW	"role-gpids/{{ssid_name}}"
Persona: MOBILITY_GW	"aaa-profile/{{ssid_name}}_<epochTimeStamp>_"

Deleting an Overlay WLAN requires three steps to be followed in sequence:

1. Delete Overlay WLAN SSID
2. Delete WLAN SSID
3. Delete Role

1. **DELETE** `{{baseUrl}}/network-config/v1/overlay-wlan/Special%20Conference`
2. **DELETE** `{{baseUrl}}/network-config/v1/wlan-ssids/Special%20Conference`
3. **DELETE** `{{baseUrl}}/network-config/v1/roles/Special%20Conference`

ONE PROBLEM CURRENTLY REMAINS

When deleting the WLAN and Role, the Policy `sys_allow_all` does NOT remove the policy assignment of the role. When deleting via GUI, it does.

This is being addressed by Engineering and should be resolved in such a way that the deletion of the WLAN also removes the rule assignment within the `sys_allow_all` policy

In both WLAN scenarios, the rule can be removed from the `sys_allow_all` policy by using the GUI to delete the WLAN. If the API has been used to remove the WLAN, you can create the WLAN with the exact name using the GUI and delete using the GUI. This will remove the rule from the `sys_allow_all` policy.

As stated previously, Engineering is working to resolve this so additional steps will no longer be necessary. Once resolved, this document will be updated to reflect any changes

[\(return to TOC\)](#)

VALIDATION

The API *Config-Health* is used to obtain the configuration health information for all devices. The request output can be filtered by using the 'filter eq' option which supports OData 4.0 on the following fields:

<i>name</i>	<i>type</i>	<i>model</i>
<i>serial</i>	<i>configStatus</i>	<i>activeIssues</i>
<i>deviceGroupName</i>	<i>deviceFunction</i>	

Some useful filters are displayed below:

- ?filter=configStatus%20eq%20'OUT_OF_SYNC'
- &filter=name%20eq%20'MARSHAAP015-02-218'
- &filter=name%20eq%20'MARSHASW08-04-IDF01'
- &filter=serial%20eq%20'TW32RDH91L'
- &filter=type%20eq%20'Switch'
- &filter=model%20eq%20'6200'
- &filter=type%20eq%20'Gateway'
- &filter=type%20eq%20'Access Point'
- &filter=model%20eq%20'AP-515-US'
- &filter=model%20eq%20'AP-505H-US'
- &filter=model%20eq%20'AP-515-RW'
- &filter=model%20eq%20'AP-505H-RW'

Examples of the API usage is displayed on the following pages.

All Device Configuration Status

In the below example, the API used obtains status of all device configurations:

GET `{{baseUrl}}/network-config/v1alpha1/config-health/devices?limit=100`

```
{
  "count": 4,
  "total": 4,
  "devices": [
    {
      "topPriorityIssue": "-",
      "lastConfigTimestamp": "1773636650975",
      "siteName": "P5-DCARW",
      "deviceGroupName": "CNX",
      "model": "AP-503H-US",
      "deployment": "Campus AP",
      "type": "Access Point",
      "role": "-",
      "serial": "CNKLEWK0L2",
      "recommendedAction": "-",
      "activeIssues": [],
      "filteredConfig": true,
      "deviceFunction": "Campus AP",
      "siteCollectionName": null,
      "name": "DCARWAP005-04-405",
      "configStatus": "SYNCHRONIZED"
    },
    {
      "topPriorityIssue": "-",
      "lastConfigTimestamp": "1773927445715",
      "siteName": "P5-DCARW",
      "deviceGroupName": "CNX",
      "model": "6300",
      "deployment": "Standalone",
      "type": "Switch",
      "role": null,
      "serial": "SG91GW501P",
      "recommendedAction": "-",
      "activeIssues": [],
      "filteredConfig": true,
      "deviceFunction": "Core Switch",
      "siteCollectionName": null,
      "name": "DCARWSW01-01-MDF",
      "configStatus": "SYNCHRONIZED"
    },
    {
      "topPriorityIssue": "-",
      "lastConfigTimestamp": "1773636647574",
      "siteName": "P5-DCARW",
      "deviceGroupName": "CNX",
      "model": "9012-US",
      "deployment": "Cluster",
      "type": "Gateway",
      "role": "Isolated Leader",
      "serial": "CNL4MCJ4L0",

```

```

        "recommendedAction": "-",
        "activeIssues": [],
        "filteredConfig": true,
        "deviceFunction": "Mobility GW",
        "siteCollectionName": null,
        "name": "DCARWGW01-01-MDF",
        "configStatus": "SYNCHRONIZED"
    },
    {
        "topPriorityIssue": "-",
        "lastConfigTimestamp": "1773927484849",
        "siteName": "P5-DCARW",
        "deviceGroupName": "CNX",
        "model": "6100",
        "deployment": "Standalone",
        "type": "Switch",
        "role": null,
        "serial": "TW22KNQ18X",
        "recommendedAction": "-",
        "activeIssues": [],
        "filteredConfig": true,
        "deviceFunction": "Access Switch",
        "siteCollectionName": null,
        "name": "DCARWSW02-01-IDF",
        "configStatus": "SYNCHRONIZED"
    }
],
"offset": 0
}

```

Filtered Device Configuration Status

In the below example, the filtering is applied to limit the responses to OUT_OF_SYNC:

GET `{{baseUrl}}/network-config/v1alpha1/config-health/devices?filter=configStatus%20eq%20'OUT_OF_SYNC'`

```
{
  "offset": 0,
  "devices": [
    {
      "deployment": "Standalone",
      "siteCollectionName": null,
      "role": null,
      "activeIssues": [
        "CONFIG_DISTRIBUTION",
        "VALIDATION_NON_BLOCKER"
      ],
      "deviceFunction": "Aggregation Switch",
      "topPriorityIssue": "Does not meet minimum firmware requirement.",
      "recommendedAction": "Upgrade the device firmware to 10.13.1110 at a minimum, or preferably to the recommended version.",
      "deviceGroupName": "SJOAP_Switches",
      "siteName": "MARSHA",
      "serial": "SG92WK38W9",
      "type": "Switch",
      "name": "MARSHASWWAN-LL-MDF",
      "configStatus": "OUT_OF_SYNC",
      "model": "6200",
      "lastConfigTimestamp": "1773943661415"
    }
  ],
  "count": 1,
  "total": 1
}
```

[\(return to TOC\)](#)

APPENDIX A – API ELEMENTS USED

Below is a list of the elements used in this Automation document.

The reference for the configuration of the API is:

Monitoring: `{{baseUrl}}/network-monitoring/v1/{{element}}/{{profile-name}}`

Configuration: `{{baseUrl}}/network-config/v1/{{element}}/{{profile-name}}`

View/Create Site List	network-config/v1/sites
Device List with scopeld	network-config/v1/devices
Device List	network-monitoring/v1/devices
Client List	network-monitoring/v1/clients

General Configuration

Admin User	network-config/v1/management-users
NTP Server	network-config/v1/ntp
DNS Server(s)	network-config/v1/dns
Create Aliases (list types of Aliases)	network-config/v1/aliases
Create VLANs	network-config/v1/layer2-vlan
Create PVLANS	network-config/v1/layer2-vlan
Create VLAN Interface IPv4	network-config/v1/vlan-interfaces
Create Syslog	network-config/v1/logging
SNMP Settings	network-config/v1/snmp
Spanning Tree	network-config/v1/stp
AAA Server	network-config/v1/auth-servers
AAA Server Group	network-config/v1/server-groups
Dot1X Profile	network-config/v1/dot1xauth
MAC Profile	network-config/v1/macauth
AAA Profile	network-config/v1/aaa-profile
Create Role if using Device Profiles	network-config/v1/roles
Create Policies if using Device Profiles	network-config/v1/policies
ACLs	network-config/v1/policies

Switch Specific

Hostname	network-config/v1/system-info?view-type=LOCAL&scope-id={{device-scope-id}}
Timezone, Loop-Protect, Global Auth	network-config/v1/switch-system
Enable Auth Globally	network-config/v1/switch-system
Login Settings (console, ssh, etc.)	network-config/v1/local-management
Create LAG	network-config/v1/portchannels?view-type=LOCAL&scope-id={{device-scope-id}}
Create Device Profile(s)	network-config/v1/device-profile
Create Port Profiles	network-config/v1/sw-port-profiles
Create VRFs	network-config/v1/vrfs
Static Route/Default Route	network-config/v1/static-route
Individual Interface Settings	network-config/v1/ethernet-interfaces?view-type=LOCAL&scope-id={{device-scope-id}}

Wireless Settings

Create Underlay WLAN	network-config/v1/wlan-ssids
Create Overlay WLAN	network-config/v1/wlan-ssids network-config/v1/overlay-wlan

Scope/Device Function Mapping

Scope Map	network-config/v1/scope-maps
-----------	------------------------------

[\(return to TOC\)](#)

APPENDIX B – NEW CENTRAL BASE URLS

Below is a list of base URLs of the various Central Clusters (API Target)

EU:	<code>https://de1.api.central.arubanetworks.com</code>
EU Central 2:	<code>https://de2.api.central.arubanetworks.com</code>
EU Central 3:	<code>https://de3.api.central.arubanetworks.com</code>
UK West 2:	<code>https://gb1.api.central.arubanetworks.com</code>
PROD:	<code>https://us1.api.central.arubanetworks.com</code>
Central PROD 2:	<code>https://us2.api.central.arubanetworks.com</code>
US West 4:	<code>https://us4.api.central.arubanetworks.com</code>
US West 5:	<code>https://us5.api.central.arubanetworks.com</code>
US East 1:	<code>https://us6.api.central.arubanetworks.com</code>
Starman:	<code>https://ca1.api.central.arubanetworks.com</code>
APAC:	<code>https://in.api.central.arubanetworks.com</code>
APAC East:	<code>https://jp1.api.central.arubanetworks.com</code>
APAC South:	<code>https://au1.api.central.arubanetworks.com</code>
UAE North 1:	<code>https://ae1.api.central.arubanetworks.com</code>
Internal 2:	<code>https://internal.api.central.arubanetworks.com</code>

[\(return to TOC\)](#)